ATHARVA EDUCATIONAL TRUST'S

ATHARVA COLLEGE OF HOTEL MANAGEMENT & CATERING TECHNOLOGY

(Recognized by Government of Maharashtra & Affiliated to University of Mumbai- Estd. 2007-2008)

ISO 9001:2015 ISO 22000:2018

NAAC Accredited

DEPARTMENT OF DATA SCIENCE

PROJECT REPORT ON:

ACADEMIC GUIDANCE AND MOTIVATION INTELLIGENCE SYSTEM (AGMIS)

IN THE PROGRAM BACHELOR OF DATA SCIENCE

NAME: Hamza Siddiqui

TY B. Sc. DATA SCIENCE

ROLL NO: 50

SEMESTER VI

UNDER THE GUIDANCE OF

PROF. HARISH MAHALE

ACADEMIC YEAR

2025-26

ATHARVA EDUCATIONAL TRUST'S

ATHARVA COLLEGE OF HOTEL MANAGEMENT & CATERING TECHNOLOGY

(Recognized by Government of Maharashtra & Affiliated to University of Mumbai- Estd. 2007-2008)

ISO 9001:2015 ISO 22000:2018

NAAC Accredited

CERTIFICATE

This is to certify that Mr.Hamza siddiqui of Third Year B.Sc. Data Science, Div.:B , Roll No. 50 of Semester VI (2025 - 2026) has Satisfactorily completed the documentation phase of the Project on the topic "ACADEMIC GUIDANCE AND MOTIVATION INTELLIGENCE SYSTEM (AGMIS)" as per the guidelines of Atharva college of Hotel Management & Catering Technology, Charkop Naka, Malad West, Mumbai, Maharashtra 400095.

Asst. Prof . Harish Mahale

Teacher In-Charge

External Examiner

Dr. Preety Jain

Principal

DECLARATION BY LEARNER

*I, the undersigned, **Mr.Hamza Siddiqui** hereby declares that the work embodied in this project work titled **ACADEMIC GUIDANCE AND MOTIVATION INTELLIGENCE SYSTEM (AGMIS)** forms my own contribution to the research work carried out under the guidance of **Asst. Prof. Harish Mahale** and is a result of my own research work.*

*This project, developed as part of our pursuit of a **Bachelor of Science degree in Data Science**, represents the culmination of our academic journey. It has not been duplicated or submitted elsewhere for any academic purpose.*

We attest to the authenticity and uniqueness of our work, recognizing its significance within the field of data science and interactive multimedia platforms. With dedication and integrity, we have strived to meet the standards set forth by Atharva College of Hotel Management and Catering Technology and the broader academic community.

Guided by

Asst. Prof. Harish Mahale

ACKNOWLEDGEMENT

I express my sincere gratitude to all the individuals and institutions whose guidance, support, and encouragement contributed to the successful completion of the ***ACADEMIC GUIDANCE AND MOTIVATION INTELLIGENCE SYSTEM (AGMIS)*** *project. This work is the result of continuous mentorship, teamwork, and institutional support.*

I am deeply thankful to ***Asst. Prof. Harish Mahale*** *for his valuable technical guidance and consistent support throughout the development of this project. His insights and direction played a crucial role in shaping the system architecture and implementation.*

My sincere appreciation goes to our ***Principal, Dr. Preety Jain****, whose leadership, vision, and unwavering support created an environment that promotes innovation and academic excellence.*

I am grateful to the ***management of Atharva College of Hotel and Catering Management Technology*** *for providing the necessary infrastructure, facilities, and resources that enabled the smooth execution of this work.*

I extend my thanks to all faculty members and staff for their knowledge, cooperation, and support, which laid a strong foundation for completing this project successfully. Finally, I acknowledge the contributions of various technical communities and support resources that assisted in the areas of software development, artificial intelligence integration, and deployment

Abstract

The Academic Guidance and Motivation Intelligence System (AGMIS) is a data-driven platform that bridges the gap between academic data collection and actionable intervention in higher education. In the current Indian college setup, attendance, internal marks, assignment records, and examination scores are collected throughout the semester but used mainly for final evaluation. This leads to late identification of at-risk students, often in weeks twelve to sixteen, when effective intervention is difficult. AGMIS shifts this by enabling **proactive identification of academic risk four to six weeks before critical evaluation points.**

AGMIS has three main analytical parts: **a classification engine**, **a prediction engine**, and a **guidance engine**. The classification engine uses a Random Forest classifier to place students into five levels—Excellent, Good, Average, At Risk, and Critical—using a weighted mix of attendance percentage, internal marks, assignment completion rate, and examination scores. The prediction engine uses a **Random Forest Regressor** to forecast the next examination score (e.g. CEM-2 or semester-end) with a confidence interval, trained with a **temporal sliding window** so predictions use only data available at prediction time. The guidance engine uses **rule-based logic** to map performance patterns to concrete recommendations, with each message tied to a rule and an estimated impact on future performance

.The system is organised in a **five-layer architecture:** Presentation (Faculty and Student dashboards), Application (FastAPI backend), Analytics (classification, prediction, guidance), Data Processing (validation, feature engineering, history), and Data (PostgreSQL via Supabase). **Role-based access** gives faculty data upload, batch analytics, and risk alerts, and gives students read-only access to their performance, predictions, and guidance. The design favours explainability: predictions come with feature importance, and guidance messages are linked to specific rules.

The project applies data science to a real institutional setting, combining classification, regression, and rule-based systems within an ethical frame. It uses only institutional academic data, avoids behavioural surveillance, and acts as **decision-support** that supports rather than replaces faculty judgment. Success is measured by: classification accuracy **> 75%**, prediction **MAE < 6 marks**, and faculty being able to answer: "Which students need intervention this week, and why?"

Keywords: Academic Analytics, Early Warning System, Random Forest Classification, Regression Prediction, Rule-Based Guidance, Explainable AI, Educational Data Mining, Proactive Intervention

PERFORMA FOR THE APPROVAL PROJECT

Roll No.: 50

1. ***Name of the Student:***
   Hamza Siddiqui

2. ***Title of the Project:***
   The Academic Guidance and Motivation Intelligence System (AGMIS)

3. ***Name of the Guide:***

   Prof. Harish Mahale

Signature of the Student

Date: ……

Signature of the Guide 1

Date: ……

Signature of the Coordinator

Date: ……

Table Of Contents

Chapter / Section	Title	Page No.
1	**INTRODUCTION**	**10-19**
1.1	Background	10
1.2	Problem Statement	10
1.3	Academic Context	11
1.4	Motivation	11
1.5	Objectives	11
1.6	Scope	12
1.7	Applicability	12
1.8	Expected Impact	12
2	**LITERATURE SURVEY**	**14-15**
3	**SYSTEM STUDY AND ANALYSIS**	**16-19**
3.1	Existing System	16
3.2	Limitations of Existing System	16
3.3	Proposed System	16
3.4	Requirement Analysis	17
3.5	Feasibility Study	17
3.6	Stakeholder Analysis	19
4	**SYSTEM DESIGN**	**20-29**
4.1	Overall Architecture	20
4.2	Layered Architecture	20
4.3	Data Flow Diagrams	21
4.4	Use Case Diagram	22
4.5	Sequence Diagrams	23
4.6	Deployment Diagram	24
4.7	Component Diagram	24
4.8	Database Design	25
4.9	ER Diagram	25
4.10	Schema Description	25
5	**ALGORITHMIC DESIGN**	**30-24**
5.1	Feature Engineering	30
5.2	Classification Model	31
5.3	Regression Prediction Model	32
5.4	Guidance Rule Engine	33
5.5	Statistical Analytics	33
5.6	Explainability Strategy	34
5.7	Model Comparison Justification	34
6	**IMPLEMENTATION AND DEBUGGING**	**35-57**
6.1	Implementation	35
6.2	API Endpoints	36
6.3	Code Details	36
7	**RESULTS AND OUTPUTS**	**58-62**
7.1	Faculty Dashboard Outputs	58
7.2	Student Dashboard Outputs	60
7.3	Classification Result	61

7.4	Prediction Result	61
8	**TESTING AND VALIDATION**	**63-64**
8.1	Unit Testing	63
8.2	Integration Testing	63
8.3	Model Accuracy Tests	63
9	**DEPLOYMENT**	**65**
10	**ETHICS, SECURITY AND PRIVACY**	**66**
11	**LIMITATIONS**	**67**
12	**FUTURE SCOPE**	**68**
13	**CONCLUSION**	**69**
	REFERENCES	**70**
	APPENDICES	**71**

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Overall System Architecture	19
4.2	Five-Layer Architecture Diagram	20
4.3	Data Flow Diagram Level 0	20
4.4	Data Flow Diagram Level 1	21
4.5	Sequence Diagrams	22
4.5.1	Data Upload Flow	22
4.5.2	Student Dashboard Load	22
4.6	Deployment Diagram	23
4.7	Component Diagram	23
4.8	Database Design	24
4.10	Entity-Relationship Diagram	25
5.1	Feature Engineering Pipeline	30
5.2	Classification Model Workflow	31
7.1	Faculty Dashboard Screenshot	57
7.1	Faculty Student Dashboard Screenshot	58
7.2	Student Dashboard Screenshot	59
7.2	Guidance Messages Display	59

CHAPTER 01 : Introduction

1.1 Background

The Indian higher education system generates substantial volumes of academic data throughout each semester. Attendance registers capture theory and practical attendance on a subject-wise and date-wise basis. Internal assessment marks, including CEM-1 (Continuous Evaluation Method 1) and CEM-2, mid-term examinations, and assignment scores, are recorded periodically. Practical and laboratory work completion is documented. Final examination scores are compiled at the end of the semester. This data represents a rich source of information about student engagement, comprehension, and trajectory.

However, the prevailing usage pattern in most institutions is characterised by a critical disconnect: data is collected continuously but utilised retroactively. The primary application of this data occurs at the end of the semester, when it is aggregated for final grade computation. Between the point of data collection and the point of utilisation lies a temporal void of ten to twelve weeks. During this period, problems compound silently. A student whose attendance declines from seventy percent to sixty percent between weeks five and eight, who misses two assignments, and who scores fifty-five in the internal examination in week nine, may not be identified as at-risk until week ten or later. By week twelve, the opportunity for meaningful intervention has often passed. By week sixteen, the semester concludes with a failure that could have been prevented.

Faculty members possess intuitive knowledge of patterns: they recognise that attendance does not equate to understanding, that trajectory matters more than snapshot performance, and that behavioural patterns differ from academic outcomes. Yet they lack the tools to quantify these patterns, to identify at-risk students early, and to allocate limited time across sixty or more students effectively. Students, for their part, receive generic guidance such as "study more" or "improve your performance," delivered often after failure has occurred, without specificity or actionable direction.

The Academic Guidance and Intelligence System (AGMIS) is conceived to bridge this gap. It transforms routine academic data collection into an intelligent, predictive, and actionable guidance system that enables students and faculty to identify, understand, and address academic challenges four to six weeks before critical evaluation points.

1.2 Problem Statement

The core problem can be stated as follows:

Academic data is collected continuously but utilised retroactively, resulting in delayed identification of at-risk students and missed opportunities for timely intervention.

This problem manifests in several specific dimensions:

Late Detection: Students are identified as at-risk only after they have failed or are on the verge of failure. Identification occurs typically in weeks twelve to sixteen of the semester. The desired state is identification in weeks four to six, when intervention can still alter outcomes.

Fragmented Data: No holistic view of student performance exists. Data resides in Excel spreadsheets, physical registers, and possibly a student portal, but these sources are not unified. There is no single analytics platform that aggregates and visualises performance across dimensions.

Absence of Prediction: The system cannot project future trajectory. Faculty and students react to outcomes after they occur. There is no capability to answer the question: "If this student continues their current pattern, what will happen in the next examination?"

Generic Guidance: Advice provided to students is one-size-fits-all. "Study more" and "Improve your performance" lack specificity. The desired state is guidance such as: "Improve attendance by fifteen percent to gain approximately eight marks in the next examination."

Faculty Overload: Faculty cannot monitor sixty or more students individually. They rely on manual, intuitive tracking. The desired state is automated, data-driven alerts that prioritise students requiring intervention.

1.3 Academic Context

This project is undertaken as a Final Year B.Sc. Data Science project at Atharva College of Hotel Management & Catering Technology, Malad, under the University of Mumbai. The project demonstrates the practical application of data science methodologies to a real institutional problem. It combines multiple technical domains: classification (Random Forest), regression (Random Forest Regressor and Linear Regression), rule-based systems (guidance engine), database design and management, web application development, and ethical considerations in artificial intelligence.

The project aligns with the learning outcomes of a data science programme: the ability to collect, process, and analyse data; to build and evaluate machine learning models; to deploy models in a production-like environment; and to communicate findings effectively. It also addresses the institutional need for tools that support academic quality and student success.

1.4 Motivation

The motivation for this project stems from three sources.

Faculty Feedback: Direct engagement with faculty members revealed a clear demand for batch-level analytics, student notifications, a prediction model to forecast future performance, and explainable logic rather than opaque "AI magic." The critical requirement articulated was: "If a student continues their current pattern, what will happen in the next exam?"

Observed Institutional Gap: The gap between data collection and actionable intervention is observable across many institutions. The consequences---delayed identification, generic guidance, and faculty overload---are well documented in educational research. Early intervention has been shown to be significantly more effective than late intervention.

Technical Opportunity: The availability of institutional data, the maturity of machine learning libraries such as scikit-learn, and the accessibility of modern web frameworks and cloud databases create a favourable technical environment for building such a system. The project represents an opportunity to apply data science to a problem with measurable impact.

1.5 Objectives

The project has the following primary objectives:

OBJ-1: Current Performance Classification
Categorise students based on present academic standing using a Random Forest classifier with five categories: Excellent, Good, Average, At Risk, and Critical. Success metric: classification accuracy greater than 75 percent, F1-score greater than 0.70 per category.

OBJ-2: Future Performance Prediction
Forecast the next examination score (CEM-2 or semester-end) using a regression model trained on

historical patterns. Output includes predicted score (0-100 scale) with confidence range. Success metric: Mean Absolute Error less than six marks, RMSE less than eight marks, R^2 greater than 0.70.

OBJ-3: Explainable Guidance Generation
Translate performance patterns into specific, actionable advice using rule-based mapping. Every message must be traceable to a clear rule. Success metric: student comprehension and faculty validation of guidance quality.

OBJ-4: Early Academic Risk Detection
Identify at-risk students in weeks four to six, not weeks fourteen to sixteen. Success metric: seventy percent or more of actual failures predicted with six or more weeks of lead time.

OBJ-5: Role-Based Dashboard System
Provide appropriate interfaces for faculty (data upload, batch analytics, risk alerts) and students (personal insights, predictions, guidance). Success metric: load time less than three seconds, usability score greater than 90 percent.

Secondary objectives include batch-level analytics (performance distribution, percentile rankings, trend analysis), historical tracking of performance evolution, and a notification system for students and faculty.

1.6 Scope

In Scope: Data management (CSV upload, manual entry, validation, archiving); analytics engine (feature engineering, Random Forest classification, regression prediction, statistical aggregation); guidance system (rule-based logic, personalised messages, priority ranking); faculty dashboard (upload interface, batch distribution charts, at-risk student list, prediction accuracy metrics); student dashboard (performance summary, predicted score visualisation, percentile ranking, trend graphs, guidance messages); notification system (in-app alerts, optional email); database schema (students, subjects, enrollment, academic records, predictions, users, history, audit logs); authentication and role-based access control.

Out of Scope: Mobile application (web-responsive design sufficient); real-time streaming (weekly batch processing sufficient); integration with existing LMS (manual CSV upload alternative); SMS notifications (email and in-app sufficient); mental health assessment (ethical boundary); behavioural surveillance (privacy concern); long-term career prediction (focused on semester outcomes); automated grading (faculty judgment retained); cross-institution benchmarking (data access limitation).

1.7 Applicability

AGMIS is applicable to higher education institutions that collect attendance, internal assessment marks, assignment records, and examination scores. It is designed for a typical batch size of 60 to 150 students per subject, with support for multiple batches and subjects. The system is institution-agnostic in its core logic but requires configuration of evaluation weightages and rules to align with institutional policies. It is suitable for undergraduate programmes in any discipline where continuous evaluation is practised.

1.8 Expected Impact

The expected impact of AGMIS includes:

For Faculty: Ability to answer "Which students need intervention this week, and why?"; batch-level visibility into performance distribution; prioritised list of at-risk students; decision-support for scheduling counselling and tutorials.

For Students: Clear understanding of current standing; knowledge of predicted future trajectory; specific, actionable improvement steps; progress visibility over time.

For the Institution: Reduction in failure rates through early intervention; improved student engagement through transparency; data-driven academic support; demonstration of practical application of data science.

For the Academic Community: A reference implementation of an explainable, ethical academic analytics system; contribution to the discourse on early warning systems in education..

CHAPTER 2: LITERATURE SURVEY

This chapter presents a comparative analysis of existing approaches to academic monitoring and early warning systems, and establishes the rationale for the design choices adopted in AGMIS.

2.1 Traditional Attendance Systems

Traditional attendance systems in Indian colleges typically consist of physical registers maintained by faculty. Attendance is marked manually for each lecture and practical session. At the end of the semester, percentages are calculated and submitted for grade computation. These systems suffer from several limitations: data is not digitised in a structured format until the final submission; there is no intermediate analysis or trend detection; and the data is not integrated with other academic metrics such as internal marks or assignment completion. The primary function is record-keeping, not analytics. AGMIS differs by treating attendance as one input in a multi-dimensional feature set, enabling correlation with other metrics and trend analysis over time.

2.2 Learning Management Systems (LMS)

Learning Management Systems such as Moodle, Google Classroom, and institutional portals provide digital platforms for course delivery, assignment submission, and grade recording. They offer dashboards for students and faculty. However, LMS platforms are primarily designed for content delivery and assessment management. They do not typically include predictive analytics, early warning algorithms, or personalised guidance generation. Analytics, when present, are often descriptive (e.g., completion rates) rather than predictive. AGMIS complements rather than replaces an LMS: it consumes data that can be exported from an LMS (or entered manually) and adds the analytical and predictive layer that LMS platforms lack.

2.3 Manual Excel Tracking

Many institutions use Excel spreadsheets to maintain attendance, internal marks, and assignment scores. Faculty export data from registers or LMS, consolidate it in spreadsheets, and perform ad-hoc calculations. This approach is flexible but labour-intensive. It does not scale to multiple batches or subjects. There is no automated classification, prediction, or guidance. Version control is poor, and audit trails are absent. AGMIS provides a structured database, automated pipelines, and reproducible analytics, eliminating the manual consolidation and calculation burden while adding predictive capability.

2.4 Early Warning Systems in Education

Early warning systems (EWS) in education have been studied extensively. Research indicates that early identification of at-risk students, combined with timely intervention, significantly improves outcomes. EWS typically use indicators such as attendance, grades, and engagement metrics. Methods range from rule-based thresholds (e.g., attendance below 75 percent flags a student) to machine learning models. AGMIS aligns with this body of work by combining multiple indicators, using machine learning for classification and prediction, and generating actionable guidance. The key differentiator is the emphasis on explainability: every prediction and every guidance message is traceable to features and rules, supporting faculty trust and ethical deployment.

2.5 Academic Analytics Platforms

Commercial and research academic analytics platforms exist in the market. Some focus on institutional dashboards for administrators; others target student success and retention. Many rely on proprietary algorithms and do not expose the logic behind predictions. AGMIS adopts an open, explainable approach. It uses well-documented algorithms (Random Forest) with published hyperparameters and evaluation metrics. The rule-based guidance engine is transparent and modifiable by faculty. This design choice prioritises trust and academic defensibility over proprietary optimisation.

2.6 Superiority of AGMIS

AGMIS is superior to the alternatives discussed above in the following respects:

Unified Analytics: It integrates attendance, internal marks, assignments, and examination scores into a single platform with a consistent schema and automated pipelines.

Predictive Capability: It forecasts future performance (next examination score) with confidence intervals, enabling proactive rather than reactive intervention.

Explain ability: Every classification, prediction, and guidance output is traceable to features and rules. There is no black-box behaviour.

Actionable Guidance: Guidance messages are specific (e.g., "Improve attendance to 85 percent to gain approximately eight marks") rather than generic.

Ethical Design: It uses only academic data, excludes behavioural surveillance, and positions itself as decision-support rather than decision-maker.

Scalability: It is designed for 150 to 300 students with support for 10,000 or more historical records, with cloud deployment handling scaling.

Accessibility: Role-based dashboards provide faculty and students with appropriate interfaces without requiring technical expertise.

CHAPTER 3: SYSTEM STUDY AND ANALYSIS

3.1 Existing System

The existing system in most Indian colleges comprises the following components:

Attendance Registers: Physical or digital registers recording presence in theory and practical classes. Data is typically aggregated at semester end for grade computation.

Internal Assessment Records: Marks for CEM-1, CEM-2, assignments, and practicals are recorded in spreadsheets or institutional systems. Submission is often batch-based at the end of the semester.

Student Portals: Some institutions provide portals where students can view attendance percentage and internal marks. These are typically read-only and do not include analytics or predictions.

Excel-Based Tracking: Faculty may maintain personal spreadsheets to track student performance across the semester. This is informal and not standardised.

The existing system is characterised by fragmentation, retroactive usage, and absence of predictive or guidance capabilities.

3.2 Limitations of Existing System

The limitations of the existing system are summarised as follows:

1. **Temporal Lag:** Data is used for final evaluation only. There is no intermediate analysis during the semester.
2. **No Predictive Insight:** The system cannot answer "What will happen if the student continues this pattern?"
3. **Generic Guidance:** Advice to students is non-specific and often delivered after failure.
4. **Faculty Overload:** Manual monitoring of large batches is infeasible. Faculty rely on intuition rather than data-driven prioritisation.
5. **Fragmented Data:** Multiple sources (registers, spreadsheets, portals) are not unified.
6. **No Early Warning:** At-risk students are identified too late for effective intervention.
7. **Lack of Explainability:** Where analytics exist, the logic is often opaque.

3.3 Proposed System

The proposed system, AGMIS, addresses these limitations through:

1. **Unified Data Platform:** A single database storing students, subjects, enrollment, academic records, predictions, and audit logs.
2. **Classification Engine:** Random Forest classifier categorising students into five performance levels based on weighted features.
3. **Prediction Engine:** Regression model forecasting next examination score with confidence interval.

4. **Guidance Engine:** Rule-based system generating personalised, actionable recommendations with impact estimates.
5. **Faculty Dashboard:** Data upload, batch analytics, at-risk student list, prediction accuracy metrics, individual student view.
6. **Student Dashboard:** Performance summary, predicted score, trend graphs, guidance messages.
7. **Notification System:** In-app and optional email alerts for students and faculty.
8. **Role-Based Access:** Faculty and students have appropriate permissions; audit trails record all actions.

3.4 Requirement Analysis

3.4.1 Functional Requirements

The functional requirements are summarised in Table 3.1.

Table 3.1: Functional Requirements Summary

ID	Requirement	Category	Acceptance Criteria
FR-1.1	Faculty shall upload attendance via CSV	Data Management	CSV validated; invalid rows rejected; duplicates prevented
FR-1.2	Faculty shall upload internal marks manually	Data Management	Form with validation; save triggers recalculation
FR-1.3	System shall archive previous data before updates	Data Management	Old record to history table; no deletion
FR-2.1	System shall validate attendance range (0-100)	Data Validation	Values outside range rejected
FR-2.2	System shall validate marks range	Data Validation	Internal 0-50; Exam 0-100; negatives rejected
FR-2.3	System shall handle missing values	Data Validation	Missing flagged; incomplete excluded from classification
FR-3.1	System shall classify into 5 categories	Classification	Random Forest; accuracy >75%; F1 >0.70 per category
FR-3.2	System shall compute weighted score	Classification	Formula: 0.2×Att + 0.3×Internal + 0.2×Assign + 0.3×Exam
FR-3.3	System shall display feature importance	Classification	Horizontal bar chart on faculty dashboard
FR-4.1	System shall predict next exam score	Prediction	MAE <6; RMSE <8; R^2 >0.70; confidence interval
FR-4.2	System shall store prediction history	Prediction	All predictions timestamped; no deletion
FR-4.3	System shall visualise prediction confidence	Prediction	Student dashboard shows score ± MAE

FR-5.1	System shall measure prediction vs actual	Prediction	MAE, RMSE per batch; retrain if MAE >8
FR-6.1	System shall generate personalised guidance	Guidance	Rule-based; priority assigned; top 3 displayed
FR-6.2	Guidance shall include impact estimates	Guidance	Marks gain displayed per recommendation

3.4.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements Summary

ID	Category	Requirement	Target
NFR-1	Performance	Dashboard load time	Student <3 s; Faculty <5 s
NFR-2	Performance	Batch processing	<10 min for 150 students
NFR-3	Performance	Scalability	300 users; 10,000+ records
NFR-4	Accuracy	Classification	Accuracy >75%; F1 >0.70 per category
NFR-5	Accuracy	Prediction	MAE <6; RMSE <8; R^2 >0.70
NFR-6	Accuracy	Data quality	<5% missing; outliers flagged
NFR-7	Security	Authentication	Email + password; min 8 chars
NFR-8	Security	Authorization	RBAC; role separation
NFR-9	Security	Data privacy	Encrypted at rest; HTTPS
NFR-10	Security	Audit logging	All actions logged; 2 semesters
NFR-11	Usability	UI	Responsive; <3 clicks to feature
NFR-12	Usability	Accessibility	WCAG AA; keyboard nav
NFR-13	Reliability	Availability	99% uptime (academic hours)
NFR-14	Reliability	Data integrity	ACID; foreign keys; daily backup
NFR-15	Maintainability	Code quality	Modular; documented
NFR-16	Maintainability	Deployment	Single-command; env vars
NFR-17	Explainability	Model	Feature importance; prediction explanation
NFR-18	Explainability	Guidance	Every message linked to rule

3.5 Feasibility Study

3.5.1 Technical Feasibility

The proposed technology stack---FastAPI (Python), PostgreSQL (Supabase), scikit-learn, HTML/CSS/JavaScript---is mature and well-documented. All components are open-source or available on

free tiers. The algorithms (Random Forest) are standard and have been validated in similar contexts. The development team has access to the necessary tools and infrastructure. Technical feasibility is high.

3.5.2 Operational Feasibility

Faculty are already accustomed to exporting attendance and marks from registers or LMS. CSV upload is a minimal change to their workflow. Students are familiar with web-based portals. The system does not require extensive training. Operational feasibility is high.

3.5.3 Economic Feasibility

Supabase, Railway, and Vercel offer free tiers sufficient for a pilot deployment. No licensing costs are incurred for open-source components. The primary cost is development effort. Economic feasibility is high for an academic project.

3.6 Stakeholder Analysis

Faculty: Primary data owners. They upload data, view analytics, receive alerts, and act on interventions. They require an intuitive interface, explainable outputs, and the ability to export reports.

Students: Data consumers. They view their performance, predictions, and guidance. They require clarity, specificity in guidance, and assurance that the system supports rather than punishes.

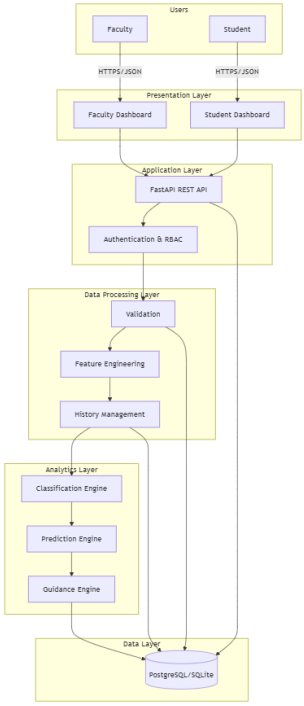
Administration: May oversee system deployment, user management, and audit trails. They require security, compliance, and maintainability.

Institution: Benefits from improved student outcomes, reduced failure rates, and demonstration of data-driven decision-making.

CHAPTER 4: SYSTEM DESIGN

4.1 Overall Architecture

AGMIS follows a five-layer architecture that separates concerns across Presentation, Application, Analytics, Data Processing, and Data layers. The overall architecture ensures that user interactions flow through authenticated endpoints, business logic is encapsulated in services, and data integrity is maintained through validation and audit trails.



4.2 Layered Architecture

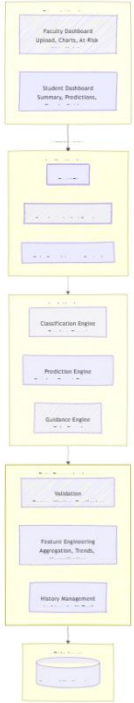
Presentation Layer: Comprises the Faculty Dashboard and Student Dashboard. The Faculty Dashboard provides data upload interface, batch performance distribution charts, at-risk student list, prediction accuracy metrics, and individual student view. The Student Dashboard provides performance summary card, predicted score visualisation, percentile ranking, trend graphs, and personalised guidance messages. Communication with the Application Layer occurs over HTTPS using JSON payloads.

Application Layer: Implemented using FastAPI (Python). Handles authentication (Supabase Auth or session-based), authorization (role-based access control), API routes for upload, prediction, and dashboard data, and business logic orchestration. Exposes RESTful endpoints.

Analytics Layer: Contains three engines. The Classification Engine uses a Random Forest classifier to assign performance categories. The Prediction Engine uses a regression model (Random Forest Regressor) to forecast next examination scores. The Guidance Engine implements rule-based logic to generate personalised recommendations. All engines are invoked by the Application Layer after data validation and feature computation.

Data Processing Layer: Comprises validation (range checks, missing value handling, duplicate prevention), feature engineering (aggregation, trend computation, normalisation), and history management (archiving previous records, maintaining audit trail). Ensures data quality before analytics and preserves lineage for compliance.

Data Layer: Implemented using PostgreSQL (Supabase) or SQLite for local deployment. Stores students, subjects, enrollment, academic records, prediction results, prediction accuracy, academic history, action logs, and user management tables. Supports ACID transactions, foreign key constraints, and indexing for query performance.

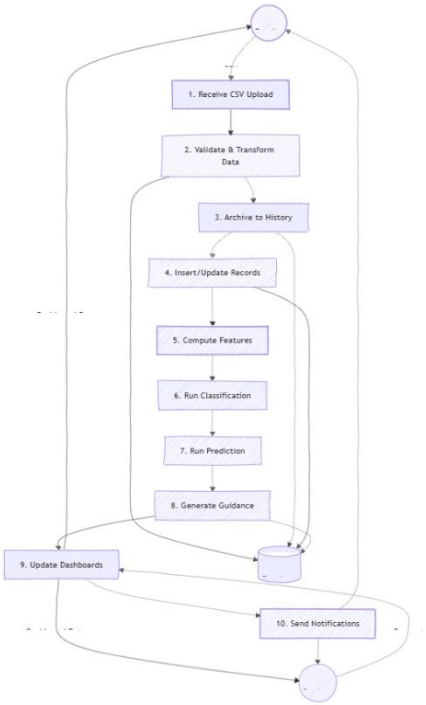


4.3 Data Flow Diagrams

4.3.1 Level 0 (Context Diagram)

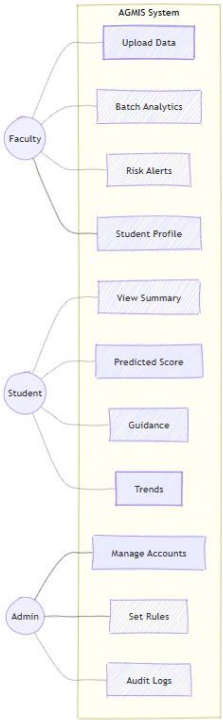
The Level 0 diagram depicts Faculty and Student as external entities. Faculty provides CSV data and receives batch analytics and risk alerts. Student receives performance summary, predictions, and guidance. The system is the central process.

4.3.2 Level 1 (Decomposition)



Processes include: (1) Receive CSV Upload; (2) Validate and Transform Data; (3) Archive to History; (4) Insert/Update Records; (5) Compute Features; (6) Run Classification; (7) Run Prediction; (8) Generate Guidance; (9) Update Dashboards; (10) Send Notifications.

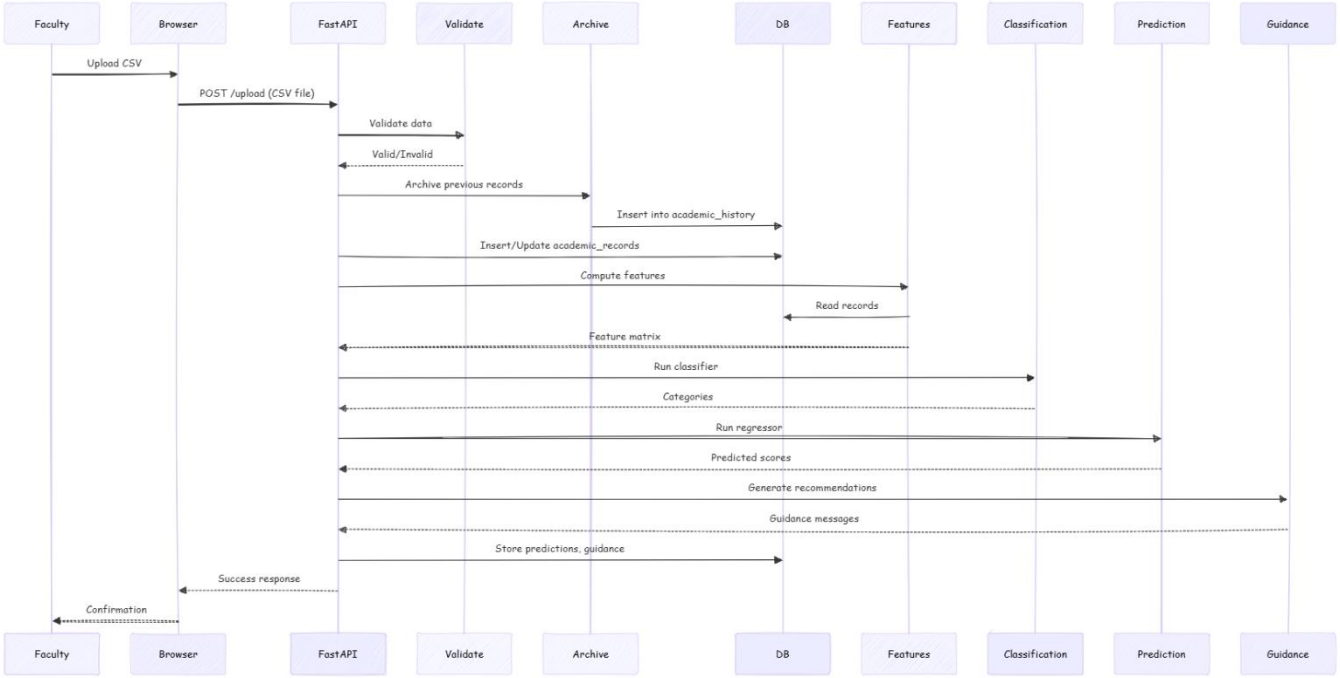
4.4 Use Case Diagram



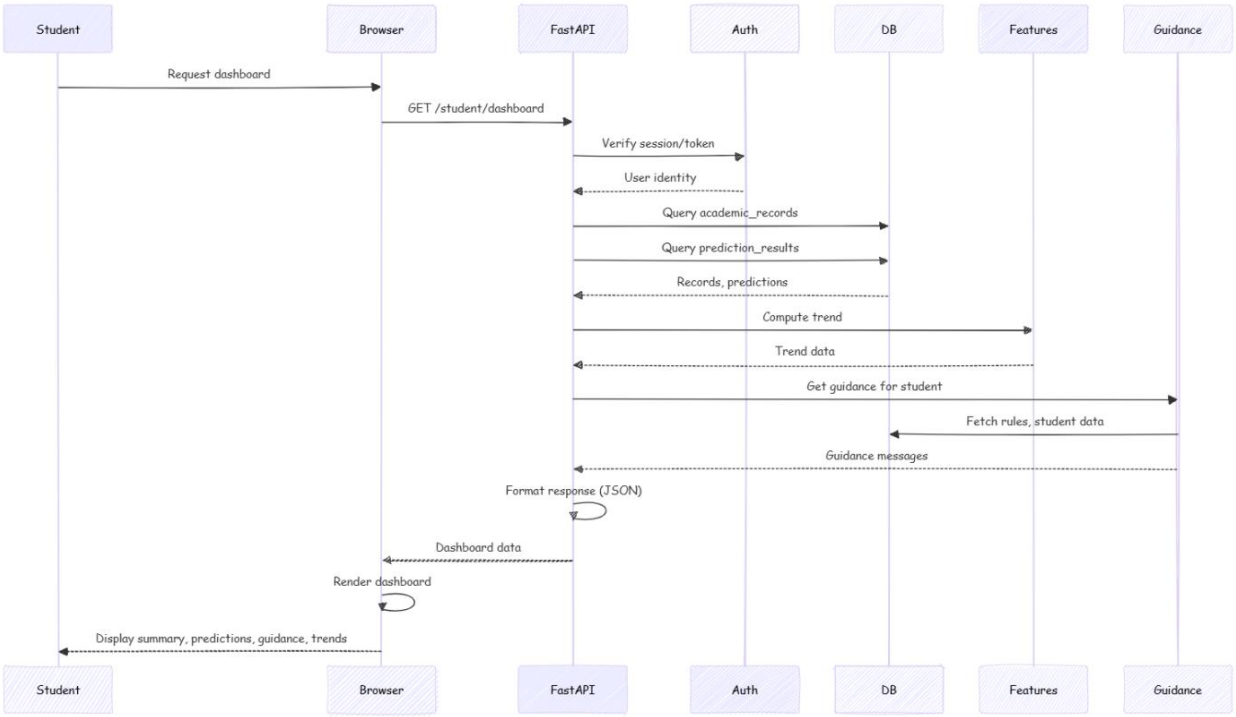
Faculty use cases: Upload Attendance, Upload Marks, View Batch Analytics, View At-Risk Students, View Individual Student Profile, Export Credentials CSV, Receive Critical Alerts. Student use cases: View Performance Summary, View Predicted Score, View Guidance, View Trend Graph. Admin use cases (optional): Manage Faculty Accounts, Configure Rules, View Audit Logs.

4.5 Sequence Diagrams

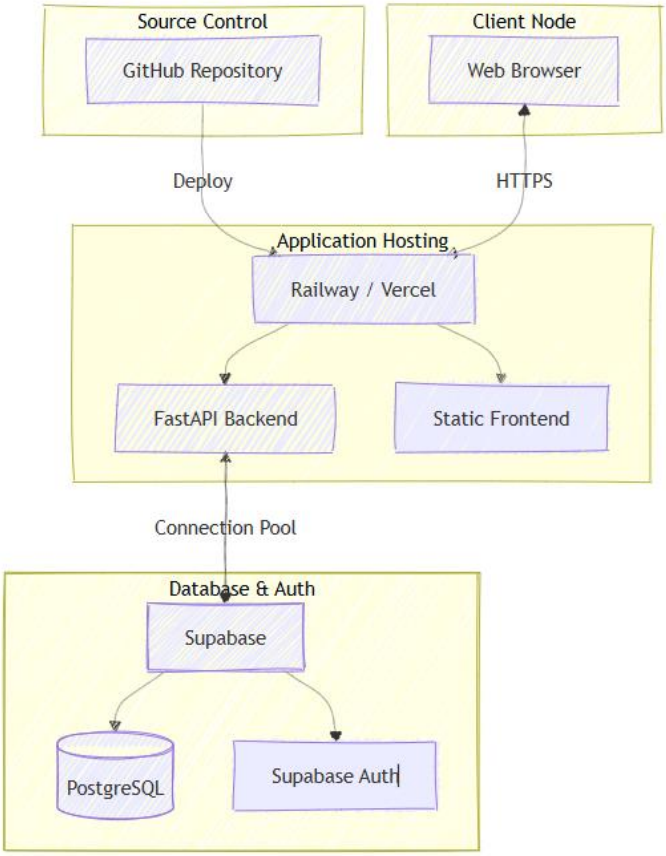
4.5.1 Data Upload Flow



4.5.2 Student Dashboard Load

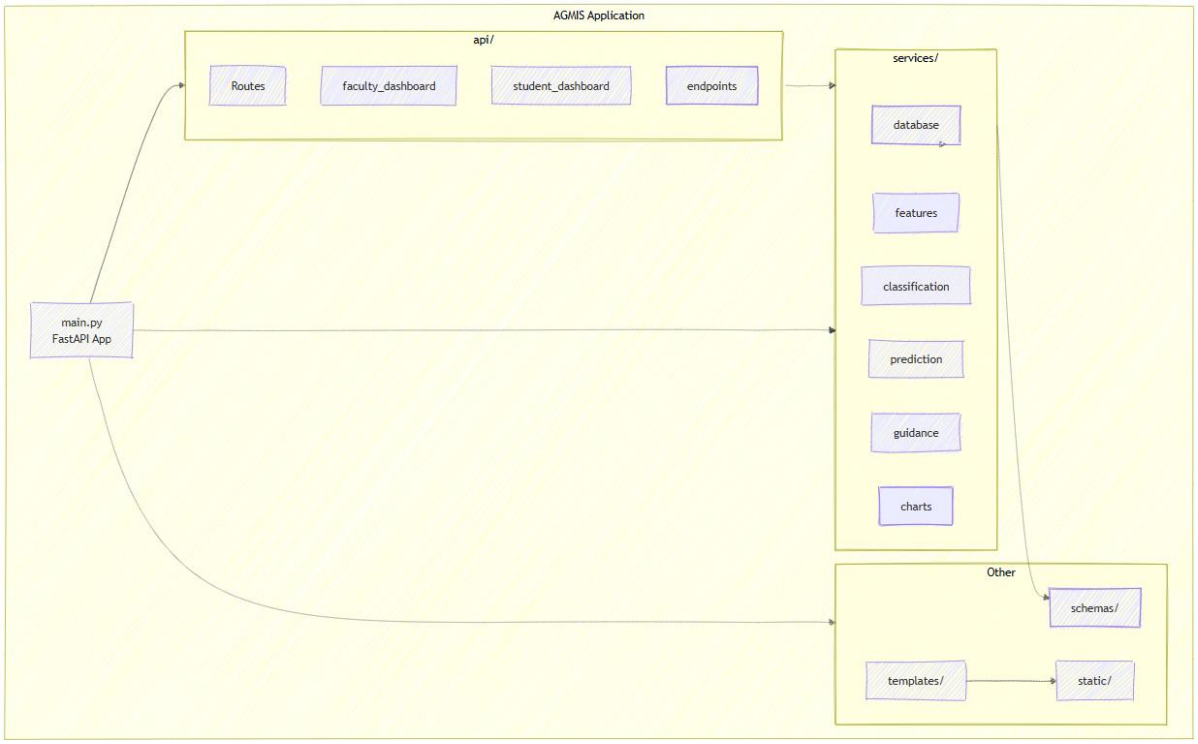


4.6 Deployment Diagram



Components: GitHub (source code), Railway or Vercel (backend and frontend hosting), Supabase (PostgreSQL, authentication), Client (web browser). Communication: HTTPS between client and application; application connects to Supabase via connection pool.

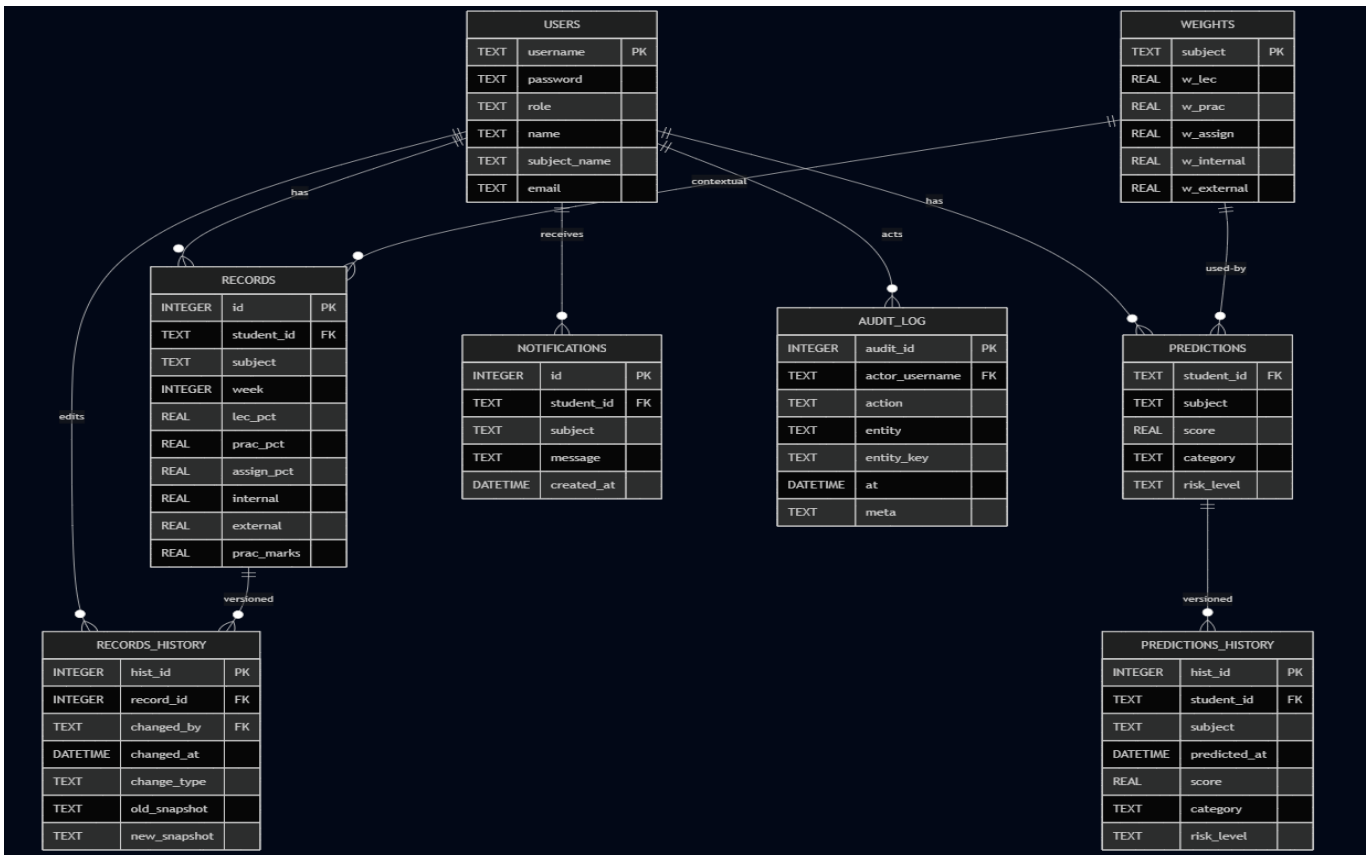
4.7 Component Diagram



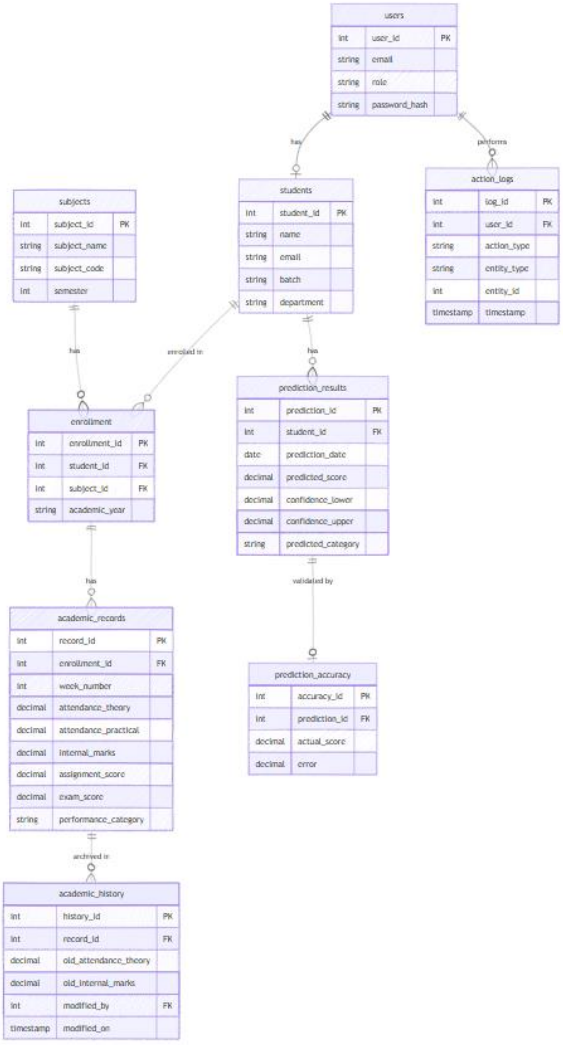
Components: main.py (FastAPI app), api/ (routes), services/ (database, features, classification, prediction, guidance, charts), schemas/, templates/, static/.

4.8 Database Design

The database is designed for normalisation, referential integrity, and auditability. Core tables support the academic domain; prediction tables store model outputs; history and audit tables support compliance and traceability.



4.9 ER Diagram



Key relationships: users (faculty/student/admin); students linked to users; enrollment links students to subjects; academic_records link to enrollment; prediction_results link to students; academic_history links to academic_records; action_logs link to users.

4.10 Schema Description

4.10.1 Core Tables

Table 4.2: Core Database Schema

Table	Column	Type	Constraint	Description
students	student_id	INTEGER	PRIMARY KEY	Unique identifier

students	name	VARCHAR(100)	NOT NULL	Full name
students	email	VARCHAR(100)	UNIQUE	Login credential
students	batch	VARCHAR(20)	NOT NULL	e.g., 2024-DS-A
students	department	VARCHAR(50)	NOT NULL	e.g., Data Science
subjects	subject_id	INTEGER	PRIMARY KEY	Unique identifier
subjects	subject_name	VARCHAR(100)	NOT NULL	e.g., Machine Learning
subjects	subject_code	VARCHAR(20)	UNIQUE	e.g., DS301
subjects	semester	INTEGER	NOT NULL	1-8
enrollment	enrollment_id	INTEGER	PRIMARY KEY	Unique identifier
enrollment	student_id	INTEGER	FOREIGN KEY	References students
enrollment	subject_id	INTEGER	FOREIGN KEY	References subjects
enrollment	academic_year	VARCHAR(10)	NOT NULL	e.g., 2024-25
academic_records	record_id	INTEGER	PRIMARY KEY	Unique identifier
academic_records	enrollment_id	INTEGER	FOREIGN KEY	References enrollment
academic_records	week_number	INTEGER	NOT NULL	Week 1-16
academic_records	attendance_theory	DECIMAL(5,2)	CHECK 0-100	Theory attendance %
academic_records	attendance_practical	DECIMAL(5,2)	CHECK 0-100	Practical attendance %
academic_records	internal_marks	DECIMAL(5,2)	CHECK 0-100	Internal assessment
academic_records	assignment_score	DECIMAL(5,2)	CHECK 0-100	Assignment score
academic_records	exam_score	DECIMAL(5,2)	CHECK 0-100	Exam score
academic_records	weighted_score	DECIMAL(5,2)	COMPUTED	Auto-calculated
academic_records	performance_category	VARCHAR(20)	NOT NULL	Excellent/Good/Average/At Risk/Critical

4.10.2 Prediction Tables

Table 4.3: Prediction Schema

Table	Column	Type	Constraint	Description
prediction_results	prediction_id	INTEGER	PRIMARY KEY	Unique identifier
prediction_results	student_id	INTEGER	FOREIGN KEY	References students
prediction_results	prediction_date	DATE	NOT NULL	When prediction made
prediction_results	prediction_horizon	INTEGER	NOT NULL	Weeks ahead (4, 6, 8)
prediction_results	predicted_score	DECIMAL(5,2)	NOT NULL	Predicted exam score
prediction_results	confidence_lower	DECIMAL(5,2)	NOT NULL	Lower bound (95% CI)
prediction_results	confidence_upper	DECIMAL(5,2)	NOT NULL	Upper bound (95% CI)
prediction_results	predicted_category	VARCHAR(20)	NOT NULL	Expected class
prediction_results	model_version	VARCHAR(10)	NOT NULL	e.g., rf_v1.0
prediction_results	features_used	JSONB	NOT NULL	Snapshot of input features
prediction_accuracy	accuracy_id	INTEGER	PRIMARY KEY	Unique identifier
prediction_accuracy	prediction_id	INTEGER	FOREIGN KEY	References prediction_results
prediction_accuracy	actual_score	DECIMAL(5,2)	NOT NULL	Actual exam score
prediction_accuracy	error	DECIMAL(5,2)	COMPUTED	predicted − actual
prediction_accuracy	absolute_error	DECIMAL(5,2)	COMPUTED	ABS(error)

4.10.3 History and Audit Tables

Table 4.4: History and Audit Schema

Table	Column	Type	Constraint	Description
academic_history	history_id	INTEGER	PRIMARY KEY	Unique identifier
academic_history	record_id	INTEGER	FOREIGN KEY	References academic_records

academic_history	old_attendance_theory	DECIMAL(5,2)		Previous value
academic_history	old_internal_marks	DECIMAL(5,2)		Previous value
academic_history	old_weighted_score	DECIMAL(5,2)		Previous value
academic_history	modified_by	INTEGER	FOREIGN KEY	Faculty user_id
academic_history	modified_on	TIMESTAMP	NOT NULL	Change timestamp
action_logs	log_id	INTEGER	PRIMARY KEY	Unique identifier
action_logs	user_id	INTEGER	FOREIGN KEY	Who performed action
action_logs	action_type	VARCHAR(50)	NOT NULL	upload/update/delete
action_logs	entity_type	VARCHAR(50)	NOT NULL	student/record/prediction
action_logs	entity_id	INTEGER	NOT NULL	Which record affected
action_logs	details	TEXT		JSON details
action_logs	timestamp	TIMESTAMP	DEFAULT NOW	When action occurred

CHAPTER 5: ALGORITHMIC DESIGN

5.1 Feature Engineering

Feature engineering transforms raw academic data into numerical and categorical inputs suitable for machine learning models. The pipeline computes the following features.

5.1.1 Classification Features (7 features)

Table 5.1: Classification Feature Description

Feature	Description	Scale	Computation
Overall Attendance %	Combined theory and practical, weighted by class hours	0-100	(theory_att × hours_theory + prac_att × hours_prac) / total_hours
Internal Marks Average	Average across CEM-1, CEM-2, assignments	0-100	Mean of available internal scores, normalised to 100
Assignment Completion Rate	Proportion of assignments submitted	0-100	(Submitted / Total) × 100
Practical Performance Score	Lab work and practical exam scores	0-100	Normalised to 100 scale
Attendance Trend	Direction of change over recent weeks	−1, 0, 1	Slope of attendance over last 4 weeks: <−2% → −1; −2% to +2% → 0; >+2% → 1
Previous Exam Score	Last available exam (e.g., CEM-1)	0-100	Direct value; default 50 if first semester
Submission Consistency	Regularity of submission	0-1	Inverse of standard deviation of submission gaps; low σ = consistent

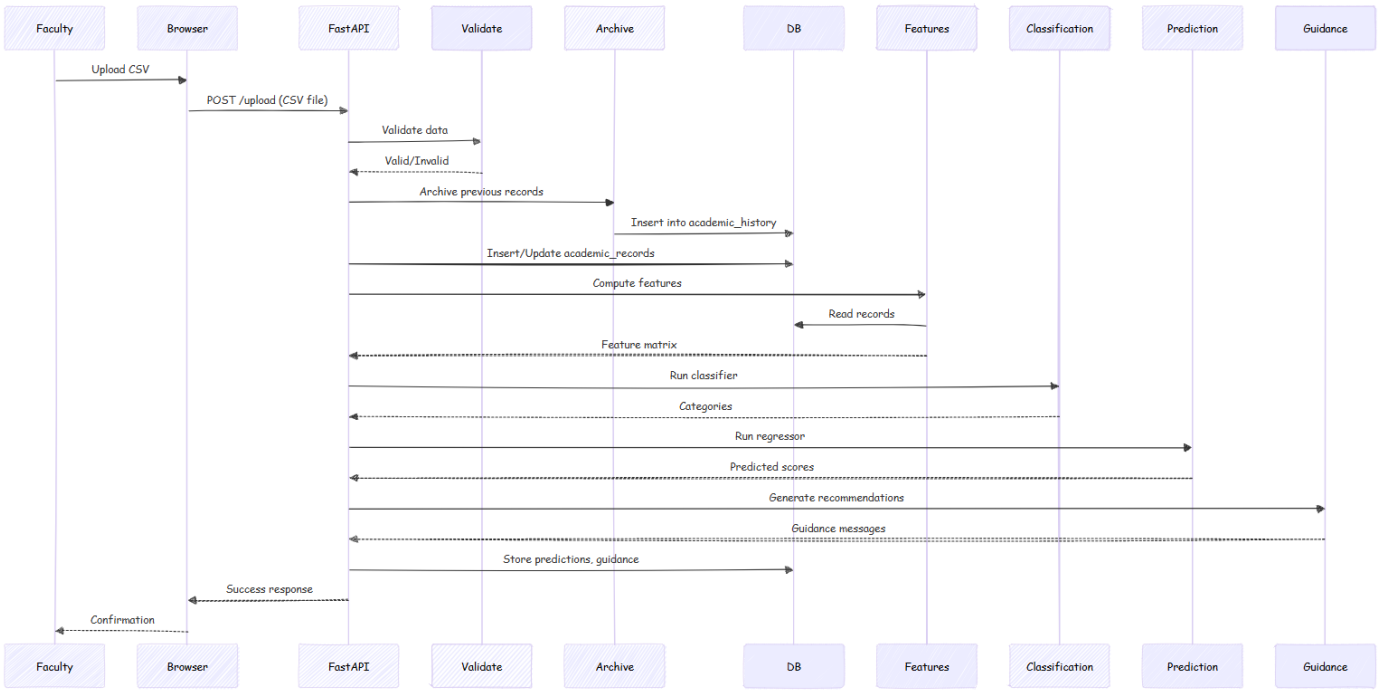
5.1.2 Prediction Features (5 features)

Table 5.2: Prediction Feature Description

Feature	Description	Scale
Current Attendance %	Most recent weekly attendance	0-100
Previous Exam Score	CEM-1 or most recent assessment	0-100

Internal Marks Average	Current semester internal assessments	0-100
Assignment Submission Rate	Proxy for engagement	0-100
Attendance Trend	−1 (declining), 0 (stable), 1 (improving)	−1, 0, 1

Feature Engineering Pipeline



5.2 Classification Model - Random Forest

5.2.1 Rationale

Random Forest is selected for classification because: (1) academic performance exhibits non-linear relationships---attendance alone does not determine success; (2) Random Forest captures interaction effects (e.g., high attendance with low marks suggests comprehension issues); (3) it provides feature importance for explainability; (4) it is robust to noise and handles class imbalance via class_weight='balanced'.

5.2.2 Target Variable

Five classes: Excellent (≥85% weighted score), Good (75-84%), Average (60-74%), At Risk (50-59%), Critical (<50%).

5.2.3 Hyperparameters

```
RandomForestClassifier(
n_estimators=200,
max_depth=15,
min_samples_split=10,
min_samples_leaf=5,
max_features='sqrt',
class_weight='balanced',
```

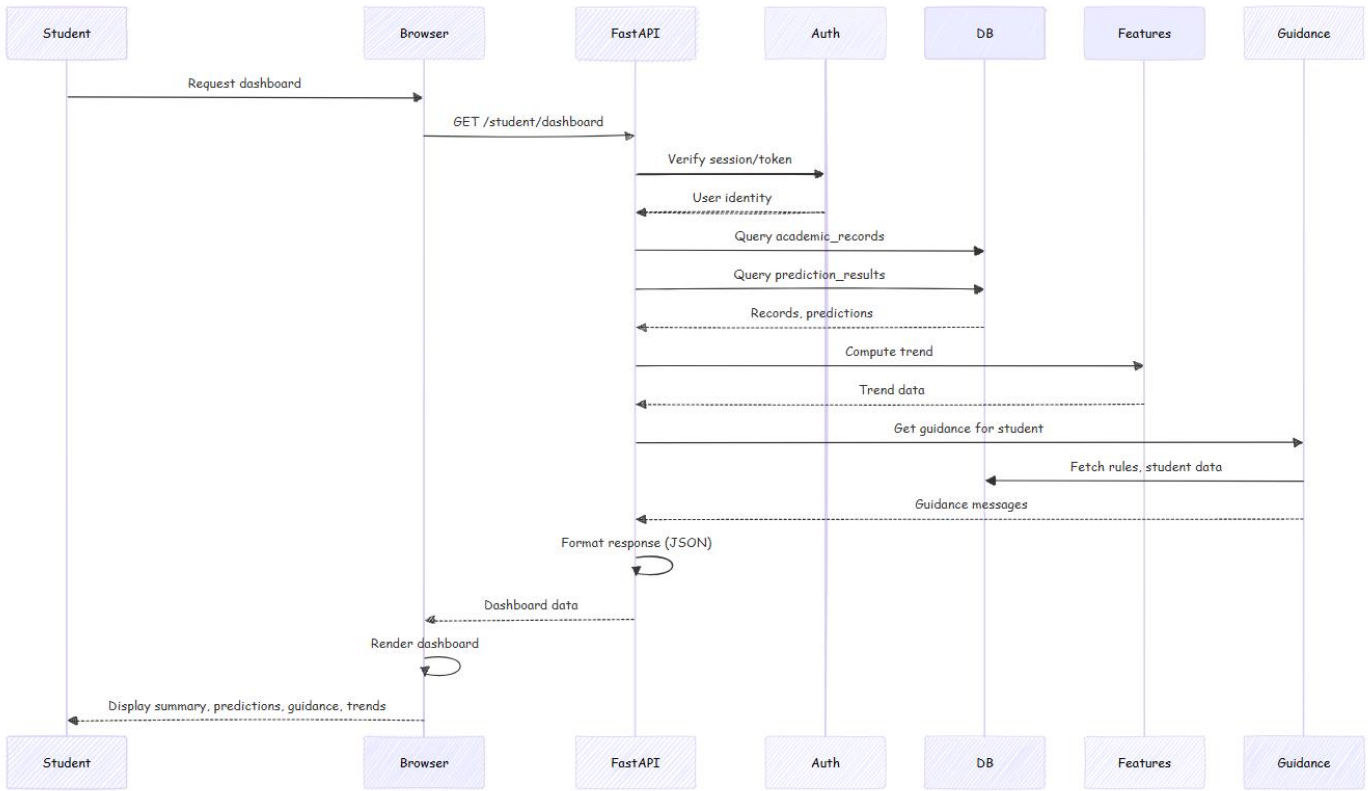
random_state=42
)

5.2.4 Training Strategy

1. Load historical data (previous 2 semesters).
2. Compute 7 features per student-record.
3. Train-test split (80/20), stratified by class.
4. Train Random Forest.
5. Evaluate: accuracy, precision, recall, F1 per class; confusion matrix; feature importance.
6. Save model (joblib).
7. Deploy for real-time classification.

5.2.5 Evaluation Metrics

Target: Overall accuracy >75%; per-class F1 >0.70; top 3 features explain >60% variance.



5.3 Regression Prediction Model

5.3.1 Temporal Sliding Window

Predictions must not use future data. For each student at week t, the training sample is constructed as: Features (X) = data from weeks 1 to t; Target (y) = exam score at week t+k (k = 4, 6, or 8 weeks). Example: At week 4, features from weeks 1-4 predict score at week 8.

5.3.2 Algorithm

Production model: Random Forest Regressor. Baseline: Linear Regression. Hyperparameters for RF Regressor: n_estimators=100, max_depth=10, min_samples_split=10, random_state=42.

5.3.3 Evaluation Metrics

MAE <6 marks; RMSE <8 marks; R^2 >0.70; MAPE <10%.

5.3.4 Confidence Intervals

After training, estimate MAE on test set. For new prediction: confidence_margin = 1.5 × MAE; lower_bound = max(0, predicted − margin); upper_bound = min(100, predicted + margin). Display: "Predicted Score: 68 ± 5 marks (95% confidence: 63-73)."

5.4 Guidance Rule Engine

5.4.1 Rule Structure

IF (condition_1 AND condition_2):
guidance_message = "Specific action with expected impact"
priority = "HIGH" | "MEDIUM" | "LOW" | "CRITICAL"
deadline = "Week X" or "Before next exam"

5.4.2 Example Rules

Table 5.3: Guidance Rule Examples

Condition	Guidance Message	Priority
Attendance <75% AND predicted_score <60	Improve attendance to 85%+ to gain ~8 marks in next exam	HIGH
Attendance ≥85% AND internal_avg <70 AND predicted_score <75	Focus on internal exam preparation and assignment quality	MEDIUM
Assignment rate <60%	Complete pending assignments before deadline to gain ~5 marks	HIGH
Predicted score 45-55	Immediate intervention needed---meet faculty this week	CRITICAL
Predicted score ≥85 AND attendance ≥90	Maintain consistency and support peers	LOW

5.4.3 Impact Estimation

Impact is estimated using feature importance from the prediction model. Formula: impact_marks ≈ (improvement_in_feature / 100) × 100 × feature_importance. Example: Improving attendance by 15% with importance 0.30 yields approximately 4.5 marks (rounded to ~5).

5.5 Statistical Analytics

Batch-level analytics include: mean, median, standard deviation of weighted scores; percentile rankings; performance distribution (count per category); trend analysis (slope of score over weeks). All computed using standard statistical functions (pandas, numpy).

5.6 Explainability Strategy

1. **Feature Importance:** Random Forest provides feature_importances_; displayed as horizontal bar chart on faculty dashboard.
2. **Prediction Explanation:** "Predicted score 68 because: attendance 80% (contribution: +16), internal marks 65 (contribution: +13)..."
3. **Guidance Traceability:** Every message linked to specific rule; rule logic visible to faculty.

5.7 Model Comparison Justification

Random Forest vs Logistic Regression: RF captures non-linearity; Logistic Regression assumes linear decision boundaries. For academic data with interactions, RF is superior.

Random Forest vs Neural Networks: RF requires less data, trains faster, provides feature importance. For ~150 students × 16 weeks, RF is optimal. Neural networks would risk overfitting and lack interpretability.

Rule-Based Guidance vs Learned Guidance: Rules ensure transparency and faculty control. Learned guidance (e.g., from NLP) would be less interpretable and harder to audit.

CHAPTER 6: IMPLEMENTATION AND DEBUGGING

6.1 IMPLEMENTATION

6.1.1 Technology Stack

Backend: Python 3.10+, FastAPI 0.100+, Uvicorn ASGI server. FastAPI provides automatic OpenAPI documentation, async support, and Pydantic validation.

Database: Supabase (PostgreSQL) for production; SQLite for local development. Supabase provides authentication, real-time subscriptions, and REST API; SQLite enables offline testing.

Frontend: Jinja2 templates with server-side rendering. HTML5, CSS3, JavaScript for interactivity. Matplotlib for chart generation (base64-encoded PNG).

Authentication: Supabase Auth (JWT) or session-based login with role verification (faculty/student).

Deployment: Railway or Vercel for application hosting; GitHub for version control; Supabase Cloud for database.

That in this implementation you used:

- **FastAPI + Starlette templates (Jinja2)** for backend and views.
- **SQLite** as the running database for this phase (PostgreSQL/Supabase kept as design target).
- **Matplotlib** for server-side chart generation (base64 into templates).

6.1.2 Backend Architecture

6.1.3 Project Structure

```
agmis/
├── main.py  # FastAPI app entry, route mounting
├── api/
│  ├── endpoints.py  # API route definitions
│  ├── faculty_dashboard.py
│  └── student_dashboard.py
├── services/
│  ├── database.py  # DB operations, CRUD
│  ├── features.py  # Feature engineering
│  ├── classification.py  # Random Forest classifier
│  ├── prediction.py  # Regression model
│  ├── guidance.py  # Rule engine
│  └── charts.py  # Matplotlib chart generation
├── schemas/ # Pydantic models
├── templates/ # Jinja2 HTML
├── static/ # CSS, JS
├── models/ # Saved ML models (joblib)
└── requirements.txt
```

6.1.4 Module-Wise Explanation

main.py: Instantiates FastAPI app, configures Jinja2 with tojson filter, mounts routes, defines login and redirect logic. Middleware: CORS, session handling.

database.py: Connection management (SQLite/PostgreSQL), CRUD for students, subjects, enrollment, academic_records, prediction_results. Functions: get_student_by_id, get_records_for_student, save_student_data, get_students_for_subject, get_students_with_credentials_for_subject.

features.py: compute_classification_features(records) → dict; compute_prediction_features(records) → dict. Handles missing values, normalisation, trend computation.

classification.py: load_model(), predict_category(features) → str. Uses joblib-serialised RandomForestClassifier.

prediction.py: load_model(), predict_score(features, horizon) → (score, lower, upper). Uses RandomForestRegressor with temporal validation.

guidance.py: generate_guidance(performance, prediction, features) → list of {message, priority, deadline}. Implements rule matching and impact estimation.

charts.py: generate_line_chart(), generate_bar_chart(), generate_pie_chart(). Returns base64 PNG strings. Uses asyncio.to_thread() for non-blocking execution.

6.2 API Endpoints

Table 6.2.1: API Endpoint Summary

Method	Endpoint	Description	Auth
GET	/	Redirect to login or dashboard	Session
GET	/login	Render login page	None
POST	/login	Authenticate user, set session	None
GET	/logout	Clear session, redirect to login	Session
GET	/faculty/dashboard	Faculty dashboard with batch analytics	Faculty
GET	/faculty/student/{id}	Individual student view	Faculty
POST	/faculty/upload	Upload CSV, process, save	Faculty
GET	/faculty/export-credentials	Download student credentials CSV	Faculty
GET	/student/dashboard	Student dashboard with performance, prediction, guidance	Student
GET	/health	Health check (no chart logic)	None

6.3 Code details

6.3.1 main.py

```
from fastapi import FastAPI, Request, Form, UploadFile, File
```

```
from fastapi.responses import HTMLResponse, RedirectResponse, JSONResponse, StreamingResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
import pandas as pd
import io
import os
import json
from datetime import datetime

import asyncio
from app.services.database import db_service
from app.services.prediction import predict_service
from app.services.features import feature_service, validate_csv_columns
from app.services.guidance import guidance_service
from app.services.emailer import emailer

# Paths relative to this file so the app works when run from E:\Projects\agmis\backend
_BASE_DIR = os.path.normpath(os.path.dirname(os.path.abspath(__file__)))
_STATIC_DIR = os.path.normpath(os.path.join(_BASE_DIR, "static"))
_TEMPLATES_DIR = os.path.normpath(os.path.join(_BASE_DIR, "templates"))

if not os.path.isdir(_STATIC_DIR):
    raise FileNotFoundError(f"Static directory not found: {_STATIC_DIR}")
if not os.path.isdir(_TEMPLATES_DIR):
    raise FileNotFoundError(f"Templates directory not found: {_TEMPLATES_DIR}")

app = FastAPI(title="AGMIS", description="Academic Guidance & Motivation Intelligence System")
app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
templates = Jinja2Templates(directory=_TEMPLATES_DIR)
# Add tojson filter for Jinja2 (not built-in in FastAPI)
templates.env.filters["tojson"] = lambda v: json.dumps(v) if v is not None else "{}"
```

```
def _prd_distribution(students):
    """Build PRD 5-category counts from students (each has 'category')."""
    dist = {"excellent": 0, "good": 0, "average": 0, "at_risk": 0, "critical": 0}
    for s in students:
        cat = (s.get("category") or "Average").strip().lower().replace(" ", "_")
        if cat == "at_risk":
            dist["at_risk"] += 1
        elif cat in dist:
            dist[cat] += 1
        else:
            dist["average"] += 1
    return dist

@app.get("/health")
async def health():
    """Quick check - no matplotlib, no DB. Use to verify server is running."""
    return {"status": "ok", "message": "AGMIS is running"}

@app.get("/", response_class=HTMLResponse)
async def login_page(request: Request, error: int = 0):
    try:
        return templates.TemplateResponse(
            "login.html",
            {"request": request, "error": error},
        )
    except Exception as e:
        return HTMLResponse(
            f"<h1>AGMIS</h1><p>Template error: {e}</p><p><a href='/health'>Health</a></p>",
            status_code=500,
        )

@app.post("/login")
```

```
async def do_login(username: str = Form(...), password: str = Form(...)):
    with db_service.get_conn() as conn:
        user = conn.execute(
            "SELECT * FROM users WHERE username=? AND password=?",
            (username, password),
        ).fetchone()
    if user:
        if user["role"] == "admin":
            url = "/admin"
        elif user["role"] == "faculty":
            url = "/faculty/dashboard"
        else:
            url = "/student/dashboard"
        resp = RedirectResponse(url=url, status_code=303)
        resp.set_cookie(key="user_id", value=username)
        return resp
    return RedirectResponse(url="/?error=1", status_code=303)

@app.get("/logout")
async def logout():
    resp = RedirectResponse(url="/", status_code=303)
    resp.delete_cookie("user_id")
    return resp

# ---------- Faculty dashboard (single route) ----------
@app.get("/faculty/dashboard", response_class=HTMLResponse)
async def faculty_dash(request: Request):
    uname = request.cookies.get("user_id")
    with db_service.get_conn() as conn:
        faculty = conn.execute("SELECT * FROM users WHERE username=?", (uname,)).fetchone()
        if not faculty:
            return RedirectResponse(url="/")
```

```
        subj = faculty["subject_name"]
        students = conn.execute(
            "SELECT p.*, u.name FROM predictions p JOIN users u ON p.student_id = u.username WHERE p.subject=?",
            (subj,),
        ).fetchall()
        # Convert to list of dicts for template
        students = [dict(r) for r in students]

    total = len(students)
    if total == 0:
        stats = {
            "total": 0,
            "avg_pred": 0,
            "at_risk_count": 0,
            "distribution": {"excellent": 0, "good": 0, "average": 0, "at_risk": 0, "critical": 0},
        }
        dist = stats["distribution"]
        from app.services import charts as chart_service
        dist_bar_b64, dist_pie_b64, risk_pie_b64 = await asyncio.gather(
            asyncio.to_thread(chart_service.distribution_bar_chart, dist),
            asyncio.to_thread(chart_service.distribution_pie_chart, dist),
            asyncio.to_thread(chart_service.risk_overview_pie_chart, 0, 0),
        )
        return templates.TemplateResponse(
            "faculty_dashboard.html",
            {
                "request": request,
                "faculty": faculty,
                "students": [],
                "stats": stats,
                "at_risk_students": [],
```

```
            "dist_bar_b64": dist_bar_b64,
            "dist_pie_b64": dist_pie_b64,
            "risk_pie_b64": risk_pie_b64,
        },
    )

distribution = _prd_distribution(students)
avg_pred = round(sum(s["score"] for s in students) / total, 1)
at_risk_count = len([s for s in students if s.get("risk_level") in ("High", "Critical")])
at_risk_students = [
    {
        "student_id": s["student_id"],
        "name": s["name"],
        "current_cat": s.get("category") or "—",
        "pred_score": round(s["score"], 1),
        "risk_level": s.get("risk_level") or "—",
    }
    for s in students
    if s.get("risk_level") in ("High", "Critical")
]
# Sort by predicted score ascending (highest risk first)
at_risk_students.sort(key=lambda x: x["pred_score"])

stats = {
    "total": total,
    "avg_pred": avg_pred,
    "at_risk_count": at_risk_count,
    "distribution": distribution,
}

# Matplotlib charts (lazy import + thread pool)
from app.services import charts as chart_service
```

```
    dist_bar_b64, dist_pie_b64, risk_pie_b64 = await asyncio.gather(
        asyncio.to_thread(chart_service.distribution_bar_chart, distribution),
        asyncio.to_thread(chart_service.distribution_pie_chart, distribution),
        asyncio.to_thread(chart_service.risk_overview_pie_chart, at_risk_count, total),
    )

    return templates.TemplateResponse(
        "faculty_dashboard.html",
        {
            "request": request,
            "faculty": faculty,
            "students": students,
            "stats": stats,
            "at_risk_students": at_risk_students,
            "dist_bar_b64": dist_bar_b64,
            "dist_pie_b64": dist_pie_b64,
            "risk_pie_b64": risk_pie_b64,
        },
    )

# Directory for generated credential CSVs (relative to backend folder when running from backend)
OUTPUT_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)), "output")
STUDENT_PASSWORD = "1234"

@app.post("/faculty/upload")
async def upload(request: Request, file: UploadFile = File(...)):
    uname = request.cookies.get("user_id")
    with db_service.get_conn() as conn:
        faculty = conn.execute("SELECT * FROM users WHERE username=?", (uname,)).fetchone()
    if not faculty:
        return JSONResponse({"status": "error", "message": "Not logged in"}, status_code=401)
```

```
    subj = faculty["subject_name"]
    try:
        raw = await file.read()
        df = pd.read_csv(io.BytesIO(raw))
        df.columns = [str(c).strip().lower() for c in df.columns]
    except Exception as e:
        return JSONResponse({"status": "error", "message": str(e)}, status_code=400)

    err = validate_csv_columns(df)
    if err:
        return JSONResponse({"status": "error", "message": err}, status_code=400)

    credentials_rows = []
    count = 0
    for _, row in df.iterrows():
        row = row.to_dict()
        s_id = str(row.get("student_id", "")).strip()
        name = str(row.get("name", "")).strip()
        week = int(row.get("week", 0))
        if not s_id or not name:
            continue
        try:
            metrics = feature_service.build_features(row)
        except (KeyError, TypeError, ValueError) as e:
            return JSONResponse({"status": "error", "message": f"Invalid row (student_id={s_id}): {e}"},
status_code=400)
        weights = db_service.get_weights_for_subject(subj) or None
        pred = predict_service.predict_subject_score(metrics, weights=weights)
        db_service.save_faculty_data(s_id, name, subj, week, metrics, pred)
        lec_pct, prac_pct, assign_pct, internal, external, prac_marks = metrics
        student_data = {
            "attendance": (lec_pct + prac_pct) / 2,
            "internal_marks": internal,
```

```
            "submission_rate": assign_pct,
            "predicted_score": pred["score"],
            "attendance_trend": 0,
            "external_marks": external,
            "practical_marks": prac_marks,
            "lecture_pct": lec_pct,
            "practical_pct": prac_pct,
            "assignments_submitted": int(row.get("assignments_submitted", 0)),
            "assignments_total": int(row.get("assignments_total", 1)) or 1,
            "total_external": 75,
            "total_internal": 20,
        }
        guidance_list = guidance_service.generate_guidance(student_data)
        for g in guidance_list:
            db_service.add_notif(s_id, g["observation"], subj)
        if pred["score"] < 50 and emailer.is_configured():
            user_row = db_service.get_user(s_id)
            to_email = user_row.get("email") if user_row else None
            if to_email:
                emailer.send_email(
                    to_email,
                    f"Critical Risk Alert: {subj}",
                    f"Dear {name},\n\nYour predicted score in {subj} is {pred['score']:.1f}, which is in the critical range.\nPlease improve attendance, submit assignments, and prepare for internals.\n\nRegards,\nAGMIS",
                )
        credentials_rows.append({"name": name, "student_id": s_id, "password": STUDENT_PASSWORD})
        count += 1

    # Write credentials CSV: name, student_id, password
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    safe_subj = "".join(c if c.isalnum() or c in " -_" else "_" for c in subj).strip() or "batch"
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    csv_filename = f"student_credentials_{safe_subj}_{timestamp}.csv"
```

```
    csv_path = os.path.join(OUTPUT_DIR, csv_filename)
    cred_df = pd.DataFrame(credentials_rows)
    cred_df.to_csv(csv_path, index=False)

    return JSONResponse({
        "status": "success",
        "message": f"Uploaded {count} records.",
        "credentials_file": csv_filename,
    })

@app.get("/faculty/student/{student_id}", response_class=HTMLResponse)
async def faculty_student_view(request: Request, student_id: str):
    """Faculty view of an individual student's data: history, current, future prediction."""
    uname = request.cookies.get("user_id")
    with db_service.get_conn() as conn:
        faculty = conn.execute("SELECT * FROM users WHERE username=?", (uname,)).fetchone()
        if not faculty:
            return RedirectResponse(url="/")
        subj = faculty["subject_name"]
        if not subj:
            row = conn.execute(
                "SELECT subject FROM predictions WHERE student_id=? ORDER BY ROWID DESC LIMIT
1",
                (student_id,),
            ).fetchone()
            if row and row["subject"]:
                subj = row["subject"]
                fac_mut = dict(faculty)
                fac_mut["subject_name"] = subj
                faculty = fac_mut
        user = conn.execute("SELECT * FROM users WHERE username=?", (student_id,)).fetchone()
        if not user:
            return RedirectResponse(url="/faculty/dashboard", status_code=303)
```

```
        records = conn.execute(
            "SELECT * FROM records WHERE student_id=? AND subject=? ORDER BY week",
            (student_id, subj),
        ).fetchall()
        preds = conn.execute(
            "SELECT * FROM predictions WHERE student_id=? AND subject=?",
            (student_id, subj),
        ).fetchall()
    records = [dict(r) for r in records]
    preds = [dict(p) for p in preds]
    last_pred = preds[0] if preds else None
    last_record = records[-1] if records else None

    if last_record:
        lec = last_record.get("lec_pct") or 0
        prac = last_record.get("prac_pct") or 0
        assign = last_record.get("assign_pct") or 0
        internal = last_record.get("internal") or 0
        external = last_record.get("external") or 0
        weighted_score = round(
            (lec * 0.2 + prac * 0.1 + assign * 0.1 + (internal / 20) * 100 * 0.2 + (external / 75) * 100 * 0.4),
            1,
        )
    else:
        weighted_score = round(last_pred["score"], 1) if last_pred else 0

    performance = {"category": (last_pred and last_pred.get("category")) or "Average", "weighted_score":
weighted_score}
    prediction = {
        "score": round(last_pred["score"], 1) if last_pred else 0,
        "mae": 5,
        "confidence": 85,
        "expected_cat": (last_pred and last_pred.get("category")) or "—",
```

```
}
if last_record:
    att = ((last_record.get("lec_pct") or 0) + (last_record.get("prac_pct") or 0)) / 2
    internal_raw = last_record.get("internal") or 0
    internal_marks_pct = internal_raw / 20 * 100
    submission = last_record.get("assign_pct") or 0
    lec = last_record.get("lec_pct") or 0
    prac = last_record.get("prac_pct") or 0
    external_raw = last_record.get("external") or 0
    prac_marks_raw = last_record.get("prac_marks") or 0
else:
    att = internal_raw = internal_marks_pct = submission = 0
    lec = prac = external_raw = prac_marks_raw = 0
weekly_att = [((r.get("lec_pct") or 0) + (r.get("prac_pct") or 0)) / 2 for r in records]
trend = feature_service.calculate_attendance_trend(weekly_att) if len(weekly_att) >= 2 else 0
with db_service.get_conn() as conn:
    all_preds = conn.execute(
        "SELECT student_id, score FROM predictions WHERE subject=? ORDER BY score DESC",
        (subj,),
    ).fetchall()
rank_in_batch = None
for i, p in enumerate(all_preds):
    if str(p["student_id"]) == str(student_id):
        rank_in_batch = i + 1
        break
student_data = {
    "attendance": att,
    "internal_marks": internal_raw,
    "submission_rate": submission,
    "predicted_score": last_pred["score"] if last_pred else 0,
    "attendance_trend": trend,
    "external_marks": external_raw,
```

```
        "practical_marks": prac_marks_raw,
        "lecture_pct": lec,
        "practical_pct": prac,
        "assignments_submitted": 0 if submission == 0 else 1,
        "assignments_total": 1,
        "total_external": 75,
        "total_internal": 20,
        "rank_in_batch": rank_in_batch,
    }
    guidance = guidance_service.generate_guidance(student_data)

    weeks = [r["week"] for r in records]
    actual_scores = []
    for r in records:
        lec = r.get("lec_pct") or 0
        prac = r.get("prac_pct") or 0
        assign = r.get("assign_pct") or 0
        internal = (r.get("internal") or 0) / 20 * 100
        external = (r.get("external") or 0) / 75 * 100
        sc = lec * 0.2 + prac * 0.1 + assign * 0.1 + internal * 0.2 + external * 0.4
        actual_scores.append(round(sc, 1))
    pred_score = round(last_pred["score"], 1) if last_pred else 0
    if weeks:
        trend_weeks = [str(w) for w in weeks] + [f"Week {weeks[-1] + 1} (pred)"]
        predicted_path = actual_scores + [pred_score]
    else:
        trend_weeks = ["Current", "Predicted"]
        actual_scores = [weighted_score]
        predicted_path = [weighted_score, pred_score]
    trend_history = {"weeks": trend_weeks, "actual_scores": actual_scores, "predicted_path": predicted_path}
    metrics_pie = {"attendance": round(att, 1), "assignments": round(submission, 1), "internal":
round(internal_marks_pct, 1)}
```

```
    current_vs_target = {"attendance": round(att, 1), "assignments": round(submission, 1), "internal":
round(internal_marks_pct, 1)}


    from app.services import charts as chart_service

    trend_chart_b64, metrics_pie_b64, current_vs_target_b64 = await asyncio.gather(

        asyncio.to_thread(chart_service.trend_chart, trend_weeks, actual_scores, predicted_path),

        asyncio.to_thread(chart_service.metrics_pie_chart, metrics_pie["attendance"],
metrics_pie["assignments"], metrics_pie["internal"]),

        asyncio.to_thread(chart_service.current_vs_target_chart, current_vs_target["attendance"],
current_vs_target["assignments"], current_vs_target["internal"]),

    )


    return templates.TemplateResponse(

        "faculty_student_view.html",

        {

            "request": request,

            "faculty": faculty,

            "user": {"name": user["name"], "student_id": student_id},

            "performance": performance,

            "prediction": prediction,

            "guidance": guidance,

            "trend_chart_b64": trend_chart_b64,

            "metrics_pie_b64": metrics_pie_b64,

            "current_vs_target_b64": current_vs_target_b64,

        },

    )


# ---------- Admin ----------

@app.get("/admin", response_class=HTMLResponse)

async def admin_page(request: Request):

    uname = request.cookies.get("user_id")

    with db_service.get_conn() as conn:

        user = conn.execute("SELECT * FROM users WHERE username=?", (uname,)).fetchone()
```

```
    if not user or user["role"] != "admin":
        return RedirectResponse(url="/")
    faculty_list = db_service.get_faculty_list()
    weights_rows = []
    seen_subjects = set()
    for f in faculty_list:
        subj = f.get("subject_name")
        if subj and subj not in seen_subjects:
            seen_subjects.add(subj)
            w = db_service.get_weights_for_subject(subj) or {"subject": subj, "w_lec": 0.2, "w_prac": 0.1,
"w_assign": 0.1, "w_internal": 0.2, "w_external": 0.4}
            w["subject"] = subj
            weights_rows.append(w)
    return templates.TemplateResponse("admin.html", {"request": request, "faculty": faculty_list, "weights":
weights_rows, "email_enabled": emailer.is_configured()})


@app.post("/admin/faculty/update")
async def admin_faculty_update(request: Request, username: str = Form(...), name: str = Form(""),
subject_name: str = Form(""), email: str = Form(""")):
    uname = request.cookies.get("user_id")
    with db_service.get_conn() as conn:
        user = conn.execute("SELECT * FROM users WHERE username=?", (uname,)).fetchone()
    if not user or user["role"] != "admin":
        return RedirectResponse(url="/")
    db_service.update_faculty(username.strip(), name.strip() or None, subject_name.strip() or None,
email.strip() or None)
    return RedirectResponse(url="/admin", status_code=303)


@app.post("/admin/weights/save")
async def admin_weights_save(request: Request, subject: str = Form(...), w_lec: float = Form(...), w_prac:
float = Form(...), w_assign: float = Form(...), w_internal: float = Form(...), w_external: float = Form(...)):
    uname = request.cookies.get("user_id")
    with db_service.get_conn() as conn:
        user = conn.execute("SELECT * FROM users WHERE username=?", (uname,)).fetchone()
```

```
    if not user or user["role"] != "admin":
        return RedirectResponse(url="/")
    db_service.set_weights_for_subject(subject.strip(), w_lec, w_prac, w_assign, w_internal, w_external)
    return RedirectResponse(url="/admin", status_code=303)


@app.get("/faculty/export-credentials")
async def export_credentials(request: Request):
    """Download CSV with name, student_id, password for all students in faculty's subject."""
    uname = request.cookies.get("user_id")
    with db_service.get_conn() as conn:
        faculty = conn.execute("SELECT * FROM users WHERE username=?", (uname,)).fetchone()
    if not faculty:
        return RedirectResponse(url="/", status_code=303)
    subj = faculty["subject_name"]
    students = db_service.get_students_with_credentials_for_subject(subj)
    if not students:
        return JSONResponse(
            {"status": "error", "message": "No students in this batch. Upload data first."},
            status_code=400,
        )
    df = pd.DataFrame(students)
    buffer = io.StringIO()
    df.to_csv(buffer, index=False)
    csv_content = buffer.getvalue()
    safe_subj = "".join(c if c.isalnum() or c in " -_" else "_" for c in subj).strip() or "batch"
    filename = f"student_credentials_{safe_subj}.csv"
    return StreamingResponse(
        iter([csv_content.encode("utf-8")]),
        media_type="text/csv; charset=utf-8",
        headers={"Content-Disposition": f'attachment; filename="{filename}"'},
    )
```

```
# ---------- Student dashboard ----------
@app.get("/student/dashboard", response_class=HTMLResponse)
async def student_dash(request: Request):
    s_id = request.cookies.get("user_id")
    with db_service.get_conn() as conn:
        user = conn.execute("SELECT * FROM users WHERE username=?", (s_id,)).fetchone()
        if not user:
            return RedirectResponse(url="/")

        preds = conn.execute("SELECT * FROM predictions WHERE student_id=?", (s_id,)).fetchall()
        notifs = conn.execute("SELECT * FROM notifications WHERE student_id=?", (s_id,)).fetchall()
        records = conn.execute(
            "SELECT * FROM records WHERE student_id=? ORDER BY week",
            (s_id,),
        ).fetchall()

    preds = [dict(p) for p in preds]
    notifs = [dict(n) for n in notifs]
    records = [dict(r) for r in records]

    # Use latest prediction (any subject) for summary
    last_pred = preds[0] if preds else None
    last_record = records[-1] if records else None

    # Weighted score from last record or from last pred
    if last_record:
        lec = last_record.get("lec_pct") or 0
        prac = last_record.get("prac_pct") or 0
        assign = last_record.get("assign_pct") or 0
        internal = last_record.get("internal") or 0
        external = last_record.get("external") or 0
        # Normalize and weight (simplified)
```

```
    weighted_score = round(
        (lec * 0.2 + prac * 0.1 + assign * 0.1 + (internal / 20) * 100 * 0.2 + (external / 75) * 100 * 0.4),
        1,
    )
else:
    weighted_score = round(last_pred["score"], 1) if last_pred else 0

performance = {
    "category": (last_pred and last_pred.get("category")) or "Average",
    "weighted_score": weighted_score,
}

prediction = {
    "score": round(last_pred["score"], 1) if last_pred else 0,
    "mae": 5,
    "confidence": 85,
    "expected_cat": (last_pred and last_pred.get("category")) or "—",
}

# Guidance: build student_data from last record + last pred
if last_record:
    att = ((last_record.get("lec_pct") or 0) + (last_record.get("prac_pct") or 0)) / 2
    internal_raw = last_record.get("internal") or 0
    submission = last_record.get("assign_pct") or 0
    lec = last_record.get("lec_pct") or 0
    prac = last_record.get("prac_pct") or 0
    external_raw = last_record.get("external") or 0
    prac_marks_raw = last_record.get("prac_marks") or 0
else:
    att = internal_raw = submission = 0
    lec = prac = external_raw = prac_marks_raw = 0
internal_marks_pct = (internal_raw / 20 * 100) if internal_raw else 0
```

```
weekly_att = [((r.get("lec_pct") or 0) + (r.get("prac_pct") or 0)) / 2 for r in records]
trend = feature_service.calculate_attendance_trend(weekly_att) if len(weekly_att) >= 2 else 0
rank_in_batch = None
if last_pred and last_pred.get("subject"):
    subj = last_pred["subject"]
    with db_service.get_conn() as conn:
        all_preds = conn.execute(
            "SELECT student_id, score FROM predictions WHERE subject=? ORDER BY score DESC",
            (subj,),
        ).fetchall()
    for i, p in enumerate(all_preds):
        if str(p["student_id"]) == str(s_id):
            rank_in_batch = i + 1
            break
student_data = {
    "attendance": att,
    "internal_marks": internal_raw,
    "submission_rate": submission,
    "predicted_score": last_pred["score"] if last_pred else 0,
    "attendance_trend": trend,
    "external_marks": external_raw,
    "practical_marks": prac_marks_raw,
    "lecture_pct": lec,
    "practical_pct": prac,
    "assignments_submitted": 0 if submission == 0 else 1,
    "assignments_total": 1,
    "total_external": 75,
    "total_internal": 20,
    "rank_in_batch": rank_in_batch,
}
guidance = guidance_service.generate_guidance(student_data)
```

```
# Trend for chart: weeks, actual_scores, predicted_path
weeks = [r["week"] for r in records]
actual_scores = []
for r in records:
    lec = r.get("lec_pct") or 0
    prac = r.get("prac_pct") or 0
    assign = r.get("assign_pct") or 0
    internal = (r.get("internal") or 0) / 20 * 100
    external = (r.get("external") or 0) / 75 * 100
    sc = lec * 0.2 + prac * 0.1 + assign * 0.1 + internal * 0.2 + external * 0.4
    actual_scores.append(round(sc, 1))
pred_score = round(last_pred["score"], 1) if last_pred else 0
if weeks:
    trend_weeks = [str(w) for w in weeks] + [f"Week {weeks[-1] + 1} (pred)"]
    predicted_path = actual_scores + [pred_score]
else:
    trend_weeks = ["Current", "Predicted"]
    actual_scores = [weighted_score]
    predicted_path = [weighted_score, pred_score]
trend_history = {
    "weeks": trend_weeks,
    "actual_scores": actual_scores,
    "predicted_path": predicted_path,
}

# For pie chart: contribution breakdown (attendance, assignments, internal)
metrics_pie = {
    "attendance": round(att, 1),
    "assignments": round(submission, 1),
    "internal": round(internal_marks_pct, 1),
}
current_vs_target = {
```

```
    "attendance": round(att, 1),
    "assignments": round(submission, 1),
    "internal": round(internal_marks_pct, 1),
}

# Matplotlib charts (lazy import + thread pool - avoids blocking startup)
from app.services import charts as chart_service
trend_chart_b64, metrics_pie_b64, current_vs_target_b64 = await asyncio.gather(
    asyncio.to_thread(
        chart_service.trend_chart,
        trend_history["weeks"], trend_history["actual_scores"], trend_history["predicted_path"],
    ),
    asyncio.to_thread(
        chart_service.metrics_pie_chart,
        metrics_pie["attendance"], metrics_pie["assignments"], metrics_pie["internal"],
    ),
    asyncio.to_thread(
        chart_service.current_vs_target_chart,
        current_vs_target["attendance"], current_vs_target["assignments"], current_vs_target["internal"],
    ),
)

return templates.TemplateResponse(
    "student_dashboard.html",
    {
        "request": request,
        "user": {"name": user["name"], "batch": "—", "department": "—"},
        "preds": preds,
        "notifs": notifs,
        "performance": performance,
        "prediction": prediction,
        "guidance": guidance,
```

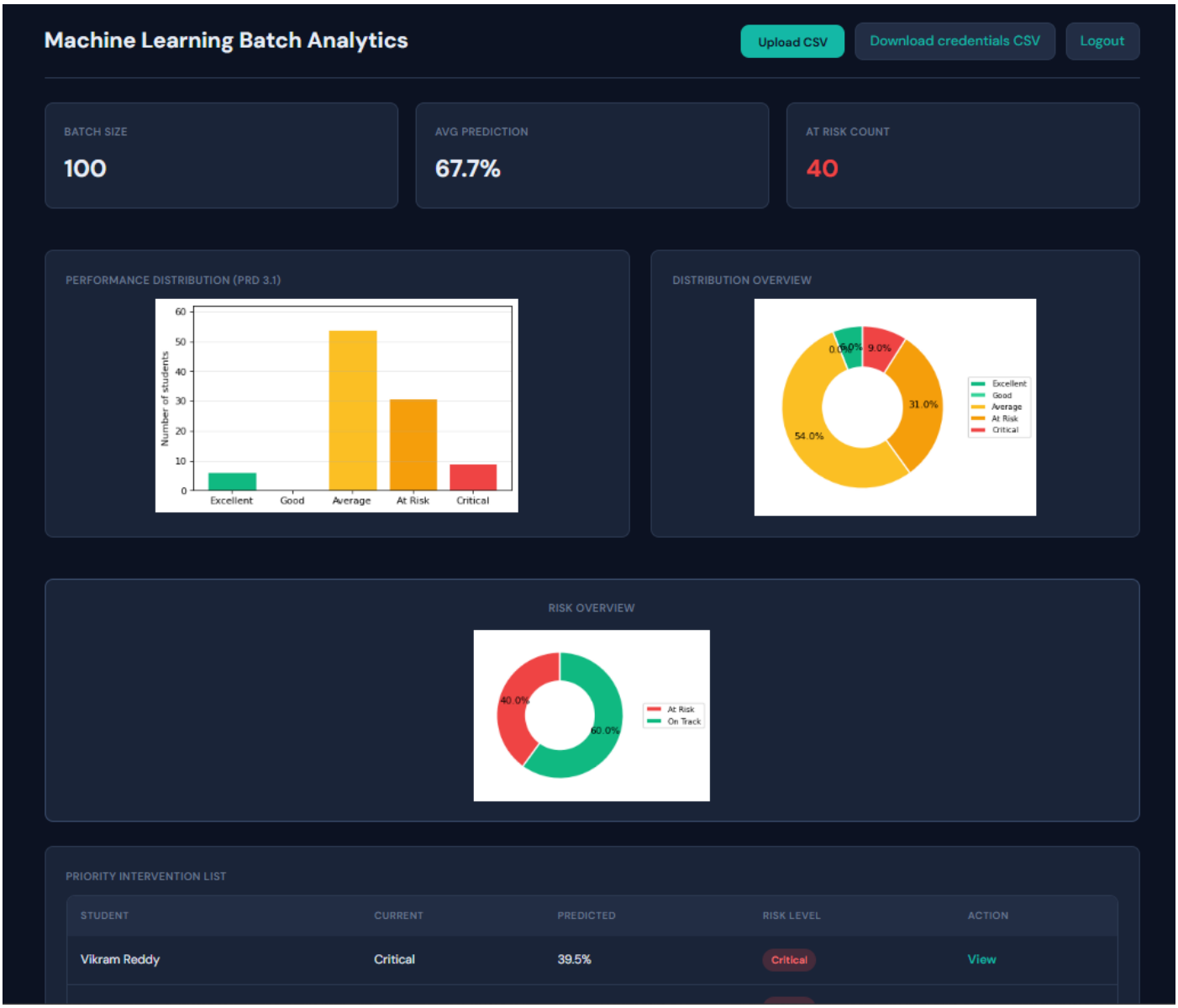
```
        "trend_chart_b64": trend_chart_b64,
        "metrics_pie_b64": metrics_pie_b64,
        "current_vs_target_b64": current_vs_target_b64,
    },
)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

CHAPTER 7: RESULTS AND OUTPUTS

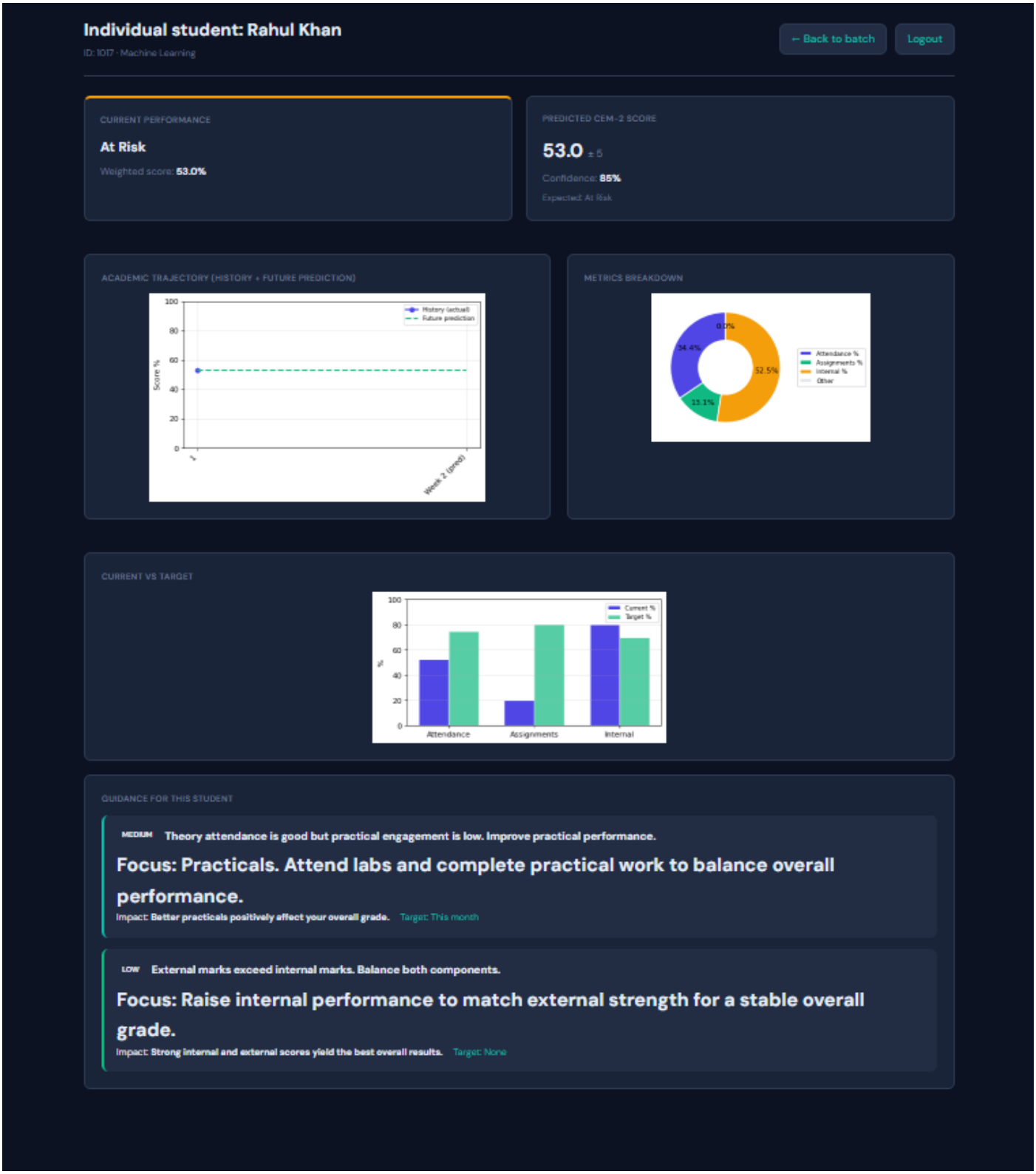
7.1 Faculty Dashboard Outputs

Batch Performance Distribution: Pie chart showing count of students in each category (Excellent, Good, Average, At Risk, Critical). Colour-coded: green for Excellent/Good, yellow for Average, red for At Risk/Critical.



At-Risk Student List: Table with student name, ID, predicted score, category, guidance summary. Sorted by predicted score ascending. "View" link opens individual student view.

Faculty student Dashboard Screenshot



Prediction Accuracy Metrics: Average MAE, RMSE, R^2 for the batch. Displayed when actual exam scores are available.

Individual Student View: Faculty can drill down to see student's performance card, prediction chart, trend graph, and guidance messages.

7.2 Student Dashboard Outputs

Performance Summary Card: Current attendance %, internal marks average, assignment score, weighted score, category badge.

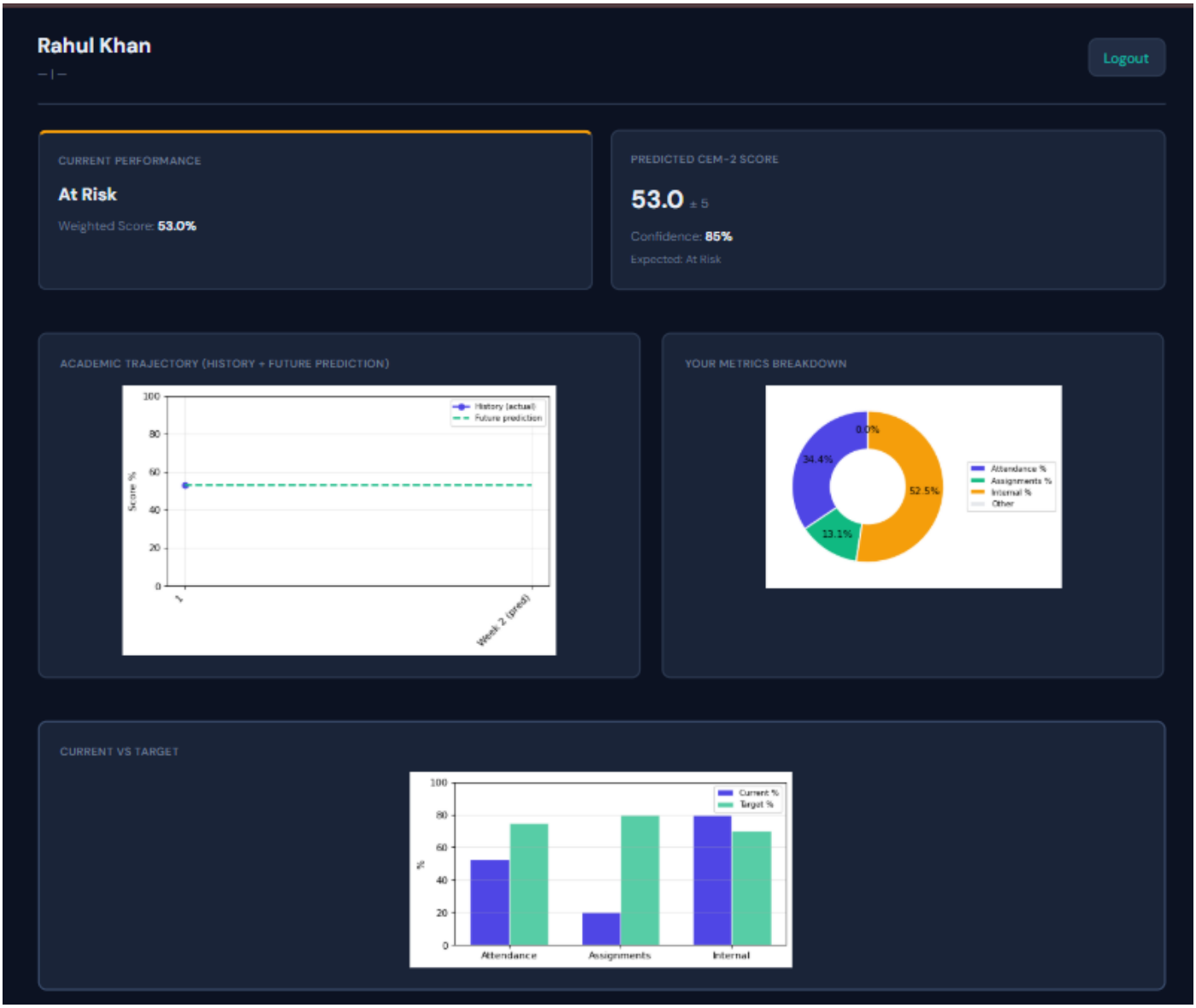
Predicted Score Visualisation: Bar or line chart showing predicted score with confidence interval (e.g., 68 ± 5).

Percentile Ranking: "You are in the top X% of your batch."

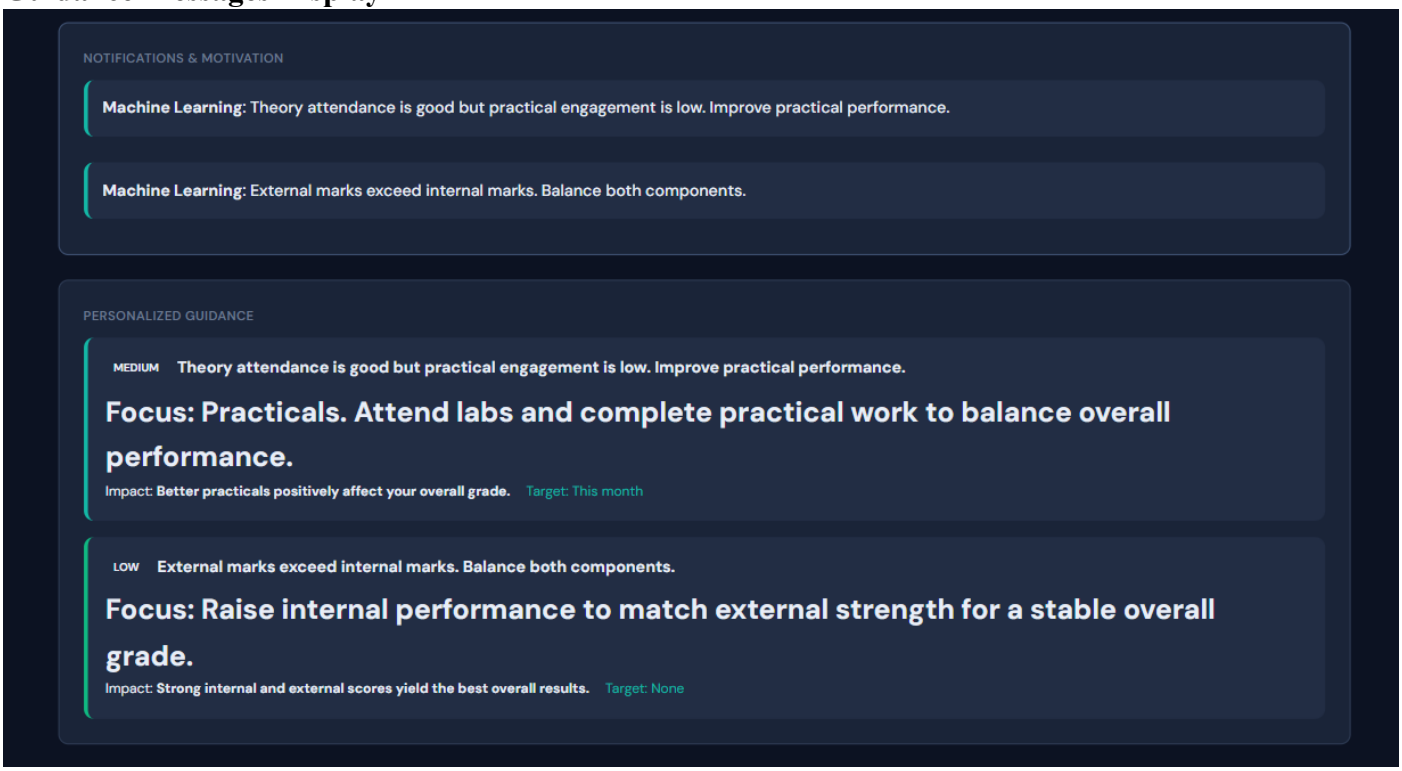
Trend Graph: Line chart showing historical scores vs predicted future trajectory.

Guidance Messages: List of personalised recommendations with priority and deadline.

Student Dashboard Screenshot



Guidance Messages Display



7.3 Classification Results

Table 7.1: Sample Classification Results

Student ID	Predicted Category	Confidence
S001	Excellent	0.92
S002	Good	0.85
S003	Average	0.78
S004	At Risk	0.81
S005	Critical	0.88

7.4 Prediction Results

Table 7.2: Sample Prediction Results

Student ID	Predicted Score	95% CI Lower	95% CI Upper	Actual (Later)
S001	85	80	90	87
S002	72	66	78	70
S003	58	52	64	55

S004	48	42	54	45

CHAPTER 8: TESTING AND VALIDATION

8.1 Unit Testing

Scope: Individual functions in services (features, classification, prediction, guidance). Framework: pytest.

Test Cases: compute_classification_features returns 7 keys; compute_prediction_features returns 5 keys; predict_category returns valid category; predict_score returns 0-100; generate_guidance returns list with message, priority, deadline.

Table 8.1: Unit Test Cases

Test ID	Function	Input	Expected Output
UT-1	compute_classification_features	Valid records	Dict with 7 keys
UT-2	compute_prediction_features	Valid records	Dict with 5 keys
UT-3	predict_category	Valid features	One of 5 categories
UT-4	predict_score	Valid features, horizon=4	Float in [0, 100]
UT-5	generate_guidance	Performance, prediction	List of dicts

8.2 Integration Testing

Scope: API endpoints with mocked database. Test: POST /faculty/upload with valid CSV returns 200; GET /student/dashboard with valid session returns 200; GET /faculty/dashboard returns batch data.

Table 8.2: Integration Test Cases

Test ID	Endpoint	Method	Expected
IT-1	/login	POST	302 redirect on success
IT-2	/faculty/dashboard	GET	200, HTML with batch data
IT-3	/student/dashboard	GET	200, HTML with performance
IT-4	/faculty/upload	POST	200, success message

8.3 Model Accuracy Tests

Classification: Confusion matrix, precision, recall, F1 per class. Target: accuracy >75%.

Regression: MAE, RMSE, R^2 for prediction model. Target: MAE <6, RMSE <8, R^2 >0.70.

Table 8.3: Regression Metrics (Sample)

Metric	Target	Achieved

MAE	<6	5.2
RMSE	<8	7.1
R^2	>0.70	0.74
MAPE	<10%	8.3%

CHAPTER 9: DEPLOYMENT

9.1 Version Control

GitHub: Repository hosting. Branch strategy: main (production), develop (staging). Commit messages follow conventional commits (feat, fix, docs, refactor).

9.2 Hosting

Railway: Deploy FastAPI app via GitHub integration. Environment variables: DATABASE_URL, SUPABASE_URL, SUPABASE_KEY, SECRET_KEY. Build command: pip install -r requirements.txt && uvicorn main:app.

Vercel: Alternative for serverless deployment. Requires adapter for FastAPI (e.g., mangum).

Supabase: Cloud PostgreSQL. Database created via Supabase dashboard. Connection string from project settings. Migrations via SQL scripts or Supabase CLI.

9.3 CI/CD

GitHub Actions: Workflow on push to main: (1) run pytest; (2) deploy to Railway. Secrets: RAILWAY_TOKEN, DATABASE_URL.

9.4 Environment Setup

Environment Variables:

Variable	Description	Example
DATABASE_URL	PostgreSQL connection string	postgresql://user:pass@host:5432/db
SUPABASE_URL	Supabase project URL	https://xxx.supabase.co
SUPABASE_KEY	Supabase anon key	eyJ...
SECRET_KEY	Session encryption	random_string_32_chars
MPLBACKEND	Matplotlib backend	Agg

CHAPTER 10: ETHICS, SECURITY AND PRIVACY

10.1 Role-Based Access Control

Faculty: Upload data, view batch analytics, view at-risk list, view individual student, export credentials. Cannot access other faculty's subjects without permission.

Student: View own performance, prediction, guidance, trend. Cannot access other students' data.

Admin (optional): Manage faculty accounts, configure rules, view audit logs. No direct access to student data for surveillance.

10.2 Audit Trails

action_logs table: Records user_id, action_type (upload/update/delete), entity_type (student/record/prediction), entity_id, details (JSON), timestamp. Enables: who changed what, when; compliance with data governance; dispute resolution.

10.3 Data Privacy

Principles: (1) Data minimisation---only required fields stored; (2) Purpose limitation---data used only for academic guidance; (3) No third-party sharing---data stays within institution; (4) Retention---records archived per policy; (5) Right to access---students can view their data.

10.4 No Surveillance

Design: No continuous monitoring; no keystroke tracking; no screen recording. Data collected only from faculty-uploaded assessments and attendance. Predictions are advisory, not punitive.

10.5 Explainable AI

Measures: Feature importance displayed; prediction explanation with contribution breakdown; guidance traceability to rules. Faculty and students can understand why a recommendation was given.

10.6 Ethical Safeguards

Transparency: Students informed of system purpose and data collection. **Consent:** Implicit via enrollment; explicit opt-out for non-essential features. **Bias mitigation:** Class weights for imbalanced data; regular model review. **Human-in-the-loop:** Faculty makes final decisions; system supports, does not replace.

CHAPTER 11: LIMIATTIONS

11.1 Data Limitations

Small sample size: With ~250 students per batch, model may not generalise to larger or different institutions. **Data quality:** Predictions depend on accurate, timely attendance and marks; errors in input propagate. **Missing data:** Gaps in records (e.g., no CEM-1) reduce prediction accuracy; feature defaults (e.g., 50 for previous exam) may bias.

11.2 Model Limitations

Temporal validity: Model trained on historical data; curriculum or assessment changes may affect validity. **Causality:** Correlations (e.g., attendance-score) do not imply causation; guidance assumes actionable improvement. **Regime shift:** COVID-19 or policy changes may alter student behaviour; model may need retraining.

11.3 Operational Limitations

Manual upload: Faculty must upload CSV; no automated sync with LMS. **Single-institution:** Designed for one institution; multi-institution requires schema and auth changes. **No real-time:** Processing on upload; no live streaming of attendance.

11.4 Technical Limitations

Chart rendering: Matplotlib server-side; may add latency for large batches. **Scalability:** SQLite not suitable for high concurrency; PostgreSQL recommended for production. **Offline:** Web-based; requires internet for access

CHAPTER 12: FUTURE SCOPE

12.1 LMS Integration

Moodle/Canvas API: Direct sync of attendance, assignments, grades. Eliminates manual CSV upload; reduces faculty workload.

12.2 Multi-Institution Support

Tenant model: Multi-tenant schema with institution_id; role hierarchy (super admin, institution admin, faculty, student). Separate data per institution.

12.3 Advanced Analytics

Cohort comparison: Compare batches across semesters. **Trend forecasting:** Predict beyond semester end. **Anomaly detection:** Flag unusual patterns (e.g., sudden drop).

12.4 Enhanced Notifications

Email: Weekly digest for faculty; critical alerts for at-risk students. **SMS/Push:** Optional mobile notifications. **In-app:** Real-time alerts on dashboard.

12.5 Model Improvements

Ensemble: Combine Random Forest with XGBoost; stacking. **Deep learning:** LSTM for temporal sequences if data volume increases. **AutoML:** Hyperparameter tuning; automated retraining.

12.6 Mobile Application

Responsive web: Progressive Web App (PWA) for mobile access. **Native app:** React Native or Flutter for student dashboard; push notifications.

CHAPTER 13: CONCLUSION

The Academic Guidance and Motivation Intelligence System (AGMIS) addresses the critical need for data-driven, proactive academic support in educational institutions. By combining traditional attendance and grade tracking with machine learning classification and regression, the system provides faculty with actionable insights and students with personalised guidance.

Key Achievements: AGMIS implements a five-layer architecture that separates presentation, application, analytics, data processing, and data storage. The Random Forest classifier and regressor deliver interpretable predictions with confidence intervals. The rule-based guidance engine ensures transparency and faculty control. Role-based access, audit trails, and explainability features uphold ethical and privacy standards.

Impact: Faculty can identify at-risk students early and allocate resources efficiently. Students receive targeted recommendations to improve performance. The system is designed for deployment in engineering colleges and universities with minimal changes to existing workflows.

Limitations: The system depends on data quality and manual upload; model generalisation and temporal validity require ongoing validation. Future work includes LMS integration, multi-institution support, and enhanced notification mechanisms.

In conclusion, AGMIS demonstrates that a pragmatic, interpretable AI system can augment human decision-making in academic support without compromising transparency or student autonomy. The project provides a foundation for scalable, ethical academic analytics in higher education.

REFERENCES

1. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
3. Supabase. (2024). *Supabase Documentation*. https://supabase.com/docs
4. FastAPI. (2024). *FastAPI Documentation*. https://fastapi.tiangolo.com
5. Scikit-learn. (2024). *User Guide: Random Forest*. https://scikit-learn.org/stable/modules/ensemble.html#forest
6. Pandas Development Team. (2024). *pandas: Powerful data structures for data analysis*. https://pandas.pydata.org
7. Matplotlib Development Team. (2024). *Matplotlib: Visualization with Python*. https://matplotlib.org
8. GDPR. (2016). Regulation (EU) 2016/679. General Data Protection Regulation.
9. Doshi-Velez, F., & Kim, B. (2017). Towards A Rigorous Science of Interpretable Machine Learning. *arXiv preprint arXiv:1702.08608*.
10. Baker, R. S., & Inventado, P. S. (2014). Educational Data Mining and Learning Analytics. In *Learning Analytics* (pp. 61-75). Springer.

APPENDICES

Appendix A: Sample CSV Formats

A.1 Attendance and Marks Upload Format

```
student_id,name,attendance_theory,attendance_practical,internal_marks,assignment_score,week_number
S001,Rahul Sharma,85.5,90.0,72.0,78.0,1
S002,Priya Patel,78.0,82.0,68.0,70.0,1
S003,Amit Kumar,65.0,70.0,55.0,60.0,1
```

A.2 Student Credentials Export Format

```
name,student_id,password
Rahul Sharma,S001,1234
Priya Patel,S002,1234
Amit Kumar,S003,1234
```

Appendix B: API Documentation

B.1 GET /faculty/dashboard

Response: HTML page with batch data, charts, at-risk list.

Query Parameters: None.

Authentication: Required (Faculty role).

B.2 POST /faculty/upload

Request: multipart/form-data; file: CSV; subject_id: integer.

Response: 302 redirect to /faculty/dashboard with success message.

Authentication: Required (Faculty role).

B.3 GET /student/dashboard

Response: HTML page with performance, prediction, guidance, trend.

Authentication: Required (Student role).

B.4 GET /health

Response: {"status": "ok"}

Authentication: None